



A Practical Guide to Direct Optimization for Planar Grid-Generation

J. E. CASTILLO*

Sandia National Laboratories, Org. 9223, MS 1110
Albuquerque, NM 87185, U.S.A.

J. S. OTTO†

Department of Computer Science, University of New Mexico
Albuquerque, NM 87131, U.S.A.

(Received and accepted August 1997)

Abstract—In this paper, we provide a guide to using the direct optimization formulation of variational grid-generation. Particular emphasis is placed on the smoothness, or length, functional; this is undoubtedly the most important functional in variational grid-generation, producing smooth grids and ensuring well-posedness of the minimization problem when combined with more unruly functionals. Unfortunately, in its most primary form, length can produce folded grids when used with nonconvex geometries. Historically, there have been two solutions to this dilemma: using an inverse mapping (the famous Winslow generator), or augmenting the functional with others that promote unicity. Both strategies have the drawback that the resulting minimization problem becomes complicated and expensive. As another alternative, we introduce a generalized strategy for length which does not use inverse mappings or auxiliary functionals, but makes strong use of reference grids. This strategy provides flexibility in controlling grid quality, and its minimization problems can be solved using a simple multigrid algorithm, yielding a robust grid-generation scheme with optimal complexity.

We also survey recent developments in the use of variational grid-generation (in the form of direct optimization) for the alignment problem. © 1999 Elsevier Science Ltd. All rights reserved.

Keywords—Variational grid-generation, Length functional, Folding, Alignment, Vector field.

1. VARIATIONAL GRID-GENERATION

Let the *physical domain* Ω be a region in the x, y -plane, and let the *logical domain* Θ be a rectangular region in the s, t -plane. Grid generators seek a bijective mapping

$$\begin{bmatrix} x(s, t) \\ y(s, t) \end{bmatrix} : \bar{\Theta} \rightarrow \bar{\Omega} \quad (1.1)$$

of the logical to the physical domain.

In practice, variational grid-generation proceeds by associating portions of the physical boundary with the sides of the logical domain, as in Figure 1, and then extending this mapping to the interior of the domains by the minimization of a functional constrained by the boundary conditions.

*This work was supported in part by a AWU/DOE Fellowship.

†This work was supported by Los Alamos National Laboratory and the Department of Energy.

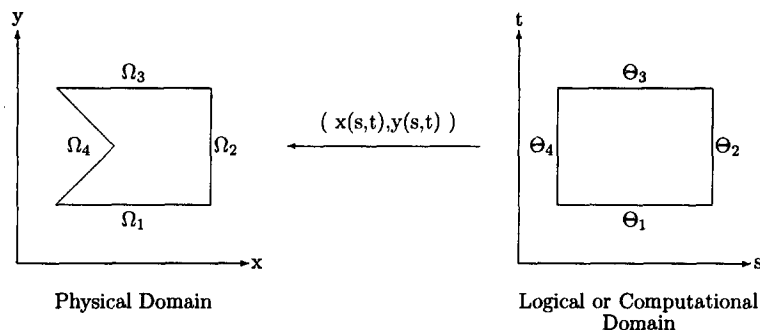


Figure 1. Physical and logical domains.

Suppose, for example, that the logical domain is the unit square. Then a canonical variational principle is to minimize the functional,

$$F(x, y) = \frac{1}{2} \int_0^1 \int_0^1 (x_s^2 + y_s^2 + x_t^2 + y_t^2) ds dt. \quad (1.2)$$

This variational principle has Euler-Lagrange equations,

$$\begin{aligned} -(x_{ss} + x_{tt}) &= 0, \\ -(y_{ss} + y_{tt}) &= 0 \end{aligned} \quad (1.3)$$

(which hints that elliptic grid-generators are a subset of variational ones). To obtain a grid, one discretizes the latter system of PDEs over a tensor-product mesh in the logical domain. Notice that the direction of the mapping has been chosen so that a boundary value problem posed in logical space is to be solved, and that one is free to choose a suitable geometry in this space. The reason for choosing a simple geometry such as a rectangle is due to the simplicity of solving boundary values problems on rectangular meshes. Not only does one want to solve the minimization problem for the grid, but typically a hosted equation in the form of a partial differential equation on the logical domain also must be solved. This is in fact one of the motivations for performing the change of coordinates: it is easy to approximate the solution of partial differential equations on rectangular meshes.

The elliptic system (1.3) represents the canonical variational grid-generator; it is intimately connected to the length or smoothness functional. The latter name stems from the above generator's notorious reputation for producing smooth grids. It has wide applicability for convex physical geometries. Unfortunately, it is also notorious for producing folded grids for nonconvex geometries. Historically, there have been two solutions to this dilemma: using an inverse mapping (the famous Winslow generator), or augmenting the functional with others that promote unicity. Both strategies have the drawback that the resulting minimization problem becomes complicated and expensive: inverse mapping replaces (1.3) with a coupled, nonlinear system, while the functionals with which length is usually augmented correspond to nonelliptic and ill-conditioned systems. Also, the strategy of using an inverse does not extend readily to three dimensions. As another alternative, we propose a general strategy for length which does not use inverse mappings or auxiliary functionals. The strategy provides flexibility in controlling grid quality, and its minimization problems can be solved using a simple multigrid algorithm, yielding a grid-generation scheme with optimal complexity.

Ideally, the work needed to compute a grid should be of the same order as that required for the solution of the hosted equation. Otherwise, this cost may outweigh the advantage gained by performing a change of coordinates. For elliptic partial differential equations, the standard of computational effort is being set by multigrid methods: under appropriate conditions one can show that the computational complexity for full multigrid is $O(N)$, where N is the size of the

grid. One of our goals has been to develop, within the framework of direct optimization, a robust set of grid-generation tools which can be used in conjunction with multigrid. In particular, the user of this set of tools should not need to be an expert in multigrid in order to achieve state-of-the-art performance. The generalized length strategy, presented here, embodies these goals. This strategy, which is presented in Sections 4 and 5, can be viewed as replacing (1.3) with a related system,

$$\begin{aligned} -(Ax_{ss} + Bx_{tt}) &= f, \\ -(Cy_{ss} + Dy_{tt}) &= g. \end{aligned} \quad (1.4)$$

The trick is to find appropriate auxiliary functions A , B , C , D , and g (which only depend on s and t). In doing this, the role of Castillo's direct optimization formulation of length [1] is key, since it treats the grid-generation problem from the beginning as discrete.

2. THE DIRECT OPTIMIZATION APPROACH

The direct optimization approach is a powerful method of variational grid-generation. Much of this power is inherited from the fact that it treats the problem of grid-generation as discrete from the start. This allows for great facility in designing and testing functionals, along with algorithms for their minimization. In addition, we will demonstrate here that equipped with some auxiliary tools for modification of the length functional this approach can be made very robust and very efficient.

Define discrete variables

$$\mathbf{x} = [x_{ij}]^T \quad \text{and} \quad \mathbf{y} = [y_{ij}]^T, \quad \text{for } 1 \leq i \leq I, \quad 1 \leq j \leq J, \quad (2.1)$$

which are in 1-to-1 association with the nodes of an $I \times J$ rectangular grid as in Figure 2.

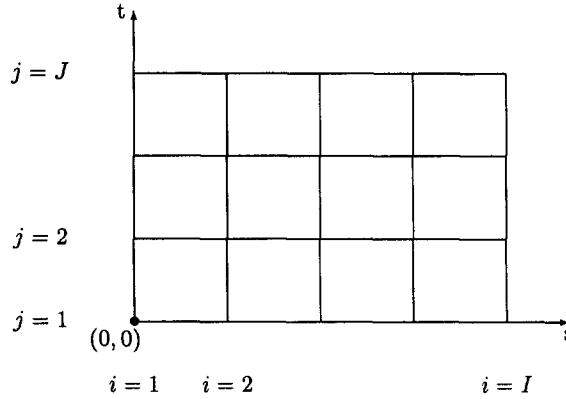


Figure 2. Logical grid.

Direct optimization begins by assigning boundary values by parametrizing boundary segments. For example, suppose in Figure 1, the physical boundary Ω_3 is given by

$$\Omega_3(s, t_J) = (x(s, t_J), y(s, t_J)), \quad 0 \leq s \leq b, \quad (2.2)$$

and then let

$$(x_{iJ}, y_{iJ}) = \Omega_3(s_i, t_J), \quad 0 = s_1 < \dots < s_I = b. \quad (2.3)$$

This defines a relation between nodes on the boundaries of the two domains. To complete the relation on the interiors, $\mathbf{x}$ and $\mathbf{y}$ are required as before to minimize a functional. The canonical functionals for direct optimization are those which control smoothness, area, and orthogonality. By far, the most important of these is the smoothness, or length, functional. The latter name

comes from the definition of the functional, i.e., as a minimizer of the variation in the length of cell edges.

Associate with each "horizontal" edge in the physical domain (see Figure 3) a length,

$$L_{ij}^h = (x_{i+1,j} - x_{ij})^2 + (y_{i+1,j} - y_{ij})^2. \quad (2.4)$$

Define lengths for the "vertical" edges similarly. Then the *vanilla length functional* is given by

$$F_L(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \sum_{E_{ij}} (L_{ij}^h)^2 + (L_{ij}^v)^2. \quad (2.5)$$

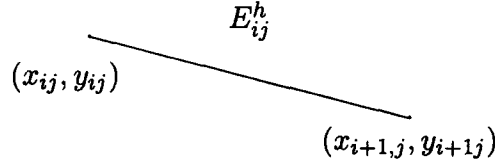


Figure 3. Horizontal edge.

In a similar manner, the area functional is designed to minimize the variation in cell areas. To define such areas, we use the notion of an oriented triangle. Following [1], let $P_{ij} = (x_{ij}, y_{ij})$, and for each physical cell C_{ij} let

$$\begin{aligned} Q &= P_{ij}, \\ R &= P_{i+1,j}, \\ S &= P_{i+1,j+1}, \\ T &= P_{i,j+1}, \end{aligned} \quad (2.6)$$

as in Figure 4. Define the area of C_{ij} as the sum,

$$A_{ij} = A(Q, R, S) + A(Q, S, T) = \frac{1}{2} \det[R - Q, S - Q] + \frac{1}{2} \det[S - Q, T - Q], \quad (2.7)$$

where $R - Q$, $S - Q$, etc., form the columns of a 2×2 matrix. The *vanilla area functional* is then given by

$$F_A(\mathbf{x}, \mathbf{y}) = \sum_{C_{ij}} A_{ij}^2, \quad (2.8)$$

where the areas are given explicitly by

$$A_{ij} = \frac{1}{2} [(x_{ij} - x_{i+1,j+1})(y_{i+1,j} - y_{i,j+1}) - (y_{ij} - y_{i+1,j+1})(x_{i+1,j} - x_{i,j+1})]. \quad (2.9)$$

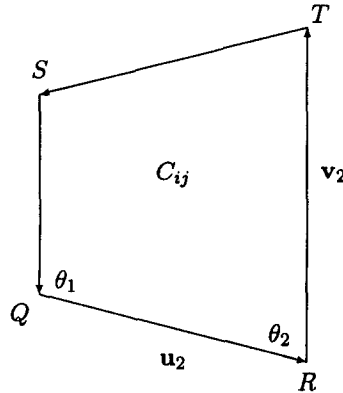


Figure 4. Physical cell C_{ij} .

As for orthogonality, for each angle θ_k in a physical cell (refer again to Figure 4) define

$$Q_{ij}^{(k)} = \langle \mathbf{u}_k, \mathbf{v}_k \rangle \quad (2.10)$$

(where $\mathbf{u}_k$ and $\mathbf{v}_k$ are the edges associated with θ_k) and

$$O_{ij} = \sum_{k=1}^4 \left(Q_{ij}^{(k)} \right)^2. \quad (2.11)$$

The *vanilla orthogonality functional* is given by

$$F_O(\mathbf{x}, \mathbf{y}) = \sum_{C_{ij}} O_{ij}. \quad (2.12)$$

Note that we have presented “vanilla” versions of the functionals, which may be associated with the generic quantities they are designed to minimize. Their definitions will be extended later—in an effort to make the functionals more robust, flavoring will be added by using weightings associated with reference grids.

We now describe how the above functionals are used in practice, along with some of their advantages and disadvantages. The following examples will emphasize the application of the functionals to nonconvex physical domains. The reason for this is that the case of a convex domain is typically an “easy” problem which can be handled by using the vanilla length functional by itself. The more interesting problems in planar grid-generation are associated with nonconvex domains.

Consider an **ear**, which is formed by halving the region bounded by a pair of concentric ellipses. Suppose we use a uniform distribution of boundary points in conjunction with the length functional as defined above. The resulting grid is shown in Figure A1. (See the Appendix for Figures A1–A25.) Folding of the grid occurs near the concave boundary edge. Now, associated with the phenomenon of folding is a change in the sign of an area of a grid cell. By derivation, the area functional minimizes the variation in area—one should be able to use area to counteract folding. Indeed, when the area functional as defined above is used for this geometry we obtain the grid in Figure A2. Although the problem with folding has been corrected, the grid is far from smooth. What happens, though, if length and area are combined? The result of such a strategy is shown in Figure A3.

The orthogonality grid for the **trapezoid** is shown in Figure A4. Even though the geometry is convex, severe distortion results from minimization of the untempered functional. In contrast, Figure A5 shows the result of using orthogonality combined with length for the same geometry.

These examples demonstrate the shortcomings that the functionals for variational grid-generation possess when used by themselves. The classical solution to this problem has been to take advantage of complementary properties of the functionals by combining them to obtain a new functional. This approach is best exemplified by area and length. Generally speaking, area produces unfolded but nonsmooth grids. Length, on the other hand, although tending to produce folded grids, is the canonical smoothness functional. To see this in the context of direct optimization, note that (2.5) is a discrete analog of (1.2). Similarly, if we form the gradient of (2.5) and set it equal to zero, we obtain a discrete version of the canonical elliptic system (1.3). A common variational strategy for nonconvex geometries, then, is to minimize some appropriate linear combination of functionals:

$$\epsilon_L \cdot F_L(\mathbf{z}) + \epsilon_A \cdot F_A(\mathbf{z}) + \epsilon_O \cdot F_O(\mathbf{z}). \quad (2.13)$$

Using such a combination, one obtains the grids of Figures A3 and A5. Although this strategy has seen wide success, it does have some drawbacks. Obviously, choosing the right parameters

is problematic. This choice is complicated by the incompatibility in the dimensionality of the functionals. This is not just a problem for direct optimization, but for variational approaches in general. Another problem has to do with the work involved in minimizing the functionals. To get a better feel for this, let us take a closer look at the Hessians associated with the vanilla functionals.

Under the lexicographical ordering of the unknowns, we have established, with the order of $\mathbf{y}$ following that of $\mathbf{x}$, the three Hessians have in common the block structure:

$$H = \begin{bmatrix} H_{xx} & H_{xy} \\ H_{xy}^\top & H_{yy} \end{bmatrix}. \quad (2.14)$$

For the vanilla functionals, one makes the following observations. For area, each block is itself a nine-point nearest neighbor stencil, giving a global 18-point stencil. For orthogonality, the structure is similar, with some additional sparsity in the off-diagonal blocks. For these two functionals, Castillo has demonstrated that there are cases where the Hessians may be nondefinite (see, for example, [2]). For length, the main diagonal blocks are five-point Laplace operators, while the off-diagonal blocks vanish.

These observations indicate that with $\epsilon_L \neq 0$, the minimization problem for (2.13) is always well-posed (i.e., a unique minimum exists). Also, despite the potential problems associated with area and orthogonality, our experience is that, in practice, the minimization problems for their functionals are often well-posed.

A robust method for the minimization of the canonical functionals of direct optimization is the Hestenes and Stiefel version of conjugate gradients (CGHS). Let

$$\mathbf{z} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix}, \quad \mathbf{g} = \nabla F(\mathbf{z}). \quad (2.15)$$

CGHS is

$$\begin{aligned} \mathbf{z}_{k+1} &= \mathbf{z}_k + \alpha_k \mathbf{d}_k, \\ \mathbf{d}_{k+1} &= -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k, \end{aligned} \quad (2.16)$$

where

$$\beta_k = \frac{\langle \mathbf{w}_k, \mathbf{g}_{k+1} \rangle}{\langle \mathbf{w}_k, \mathbf{d}_k \rangle} \quad \text{and} \quad \mathbf{w}_k = \mathbf{g}_{k+1} - \mathbf{g}_k. \quad (2.17)$$

In particular, with α_k chosen to minimize $F(\mathbf{z}_k + \alpha_k \mathbf{d}_k)$ (this is called an exact line search), we have found this to be a reliable method for minimizing these functionals. For each of the functionals described above notice that

$$F(\mathbf{z} + \alpha \mathbf{d}) = p_m(\alpha), \quad (2.18)$$

where p_m is a polynomial of degree $m = 2, 4$, and 4 , respectively, for length, area, and orthogonality. In order to minimize this quantity with respect to α_k , one simply solves $p'_m(\alpha_k) = 0$. For length, this equation is linear; for area and orthogonality we have used Newton's method to solve this equation.

Notice that evaluation of p and p' is particularly expensive for the orthogonality functional. Because of this, we have also considered economical versions of this functional. These can be obtained by restricting the set of angles over which the summation in (2.11) is performed. In particular, the "lower" version of the functional uses

$$O_{ij} = \sum_{k=1}^2 \left(Q_{ij}^{(k)} \right)^2, \quad (2.19)$$

which just sums over θ_1 and θ_2 in Figure 4. Our experience is that this version often produces grids similar to those obtained using the full version. All of the experimental results in this paper for the orthogonality functional refer to this economical version.

The spectral properties of the Hessian for length are well known, as is the convergence behavior of CGHS and multigrid. Spectral analysis of the Hessians associated with area and orthogonality proves to be more difficult. To demonstrate the convergence of CGHS, we present results of some functional minimizations for the ear. Iteration counts for each of the functionals on a succession of refined grids are given in Tables 1–3. Here, m is used to index the refinement level of a grid, the various grids having dimensions $I = J = n \equiv 2^m + 1$. Let $N = I \cdot J = n^2$, and for a given functional, let $w(N)$ be a measure of the work required for each iteration of CGHS. For length and area, the total work required is $\sqrt{N} \cdot w(N)$. For orthogonality, the total work required is at least $N \cdot w(N)$.

Table 1. Area functional.

m	$N_{\text{its}} \approx 4 \cdot 2^m$	$N = n^2 \approx 4^m$
2	13	25
3	38	81
4	59	289
5	121	1089
6	249	4225

Table 2. Orthogonality functional.

m	$N_{\text{its}} > 4^m$	$N = n^2 \approx 4^m$
2	16	25
3	81	81
4	420	289

Table 3. Length functional.

m	$N_{\text{its}} \approx 2^m$	$N = n^2 \approx 4^m$
2	5	25
3	10	81
4	19	289
5	37	1089
6	70	4225

We summarize some properties of the three functionals from the above discussion.

Area

- Area typically produces unfolded grids on nonconvex domains.
- The generated grids can be very nonsmooth.
- The work required by CG to minimize F_A is $\sqrt{N} \cdot w(N)$.
- The Hessian for F_A is nonconstant and may fail to be positive definite.
- The stencil for the Hessian is large (18-point); the major blocks are coupled.
- It is not clear how to accelerate minimization of the area functional.
- Area can be combined with the length functional to obtain unfolded grids.

Orthogonality

- The work required by CG to minimize F_O can be $N \cdot w(N)$.
- The Hessian for F_O is nonconstant and may fail to be positive definite.
- The structure of the Hessian is similar to that of the area functional.
- It is not clear how to accelerate minimization of the orthogonality functional.

Length

- Length typically produces smooth grids.
- The generated grids can fold on nonconvex domains.
- The work required by CG to minimize F_L is $\sqrt{N} \cdot w(N)$.
- The Hessian for F_L is constant and is positive definite.
- The stencil for the Hessian is small (five-point); the major blocks are decoupled.
- Minimization of the length functional can easily be accelerated using multigrid.
- Length can be combined with the area functional to smooth grids, or to make the Hessian positive definite.

3. REFERENCE GRIDS AND ELLIPTIC GRID-GENERATORS

Consider the mapping of a simply connected domain to a rectangular domain, as in Figure 5, via the elliptic system:

$$\begin{aligned} ps_{xx} + qs_{yy} &= 0, \\ pt_{xx} + qt_{yy} &= 0. \end{aligned} \quad (3.1)$$

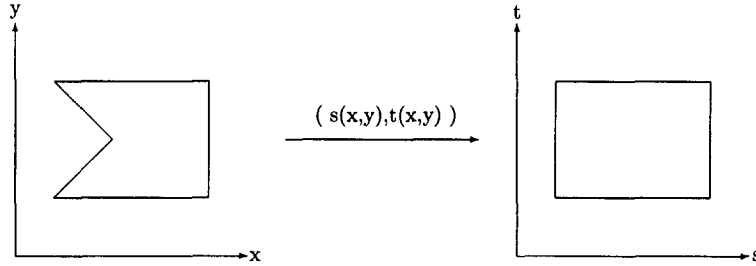


Figure 5. Mapping to a convex domain.

Since each equation satisfies a maximum principle, and the target domain is convex, the problem of folding at the boundary cannot occur. This has led to the use of the inverse mapping for (1.1). In particular, the inverse of the Laplace system is

$$\begin{aligned} \alpha x_{ss} - 2\beta x_{st} + \gamma x_{tt} + J^2 &= 0, \\ \alpha y_{ss} - 2\beta y_{st} + \gamma y_{tt} + J^2 &= 0, \end{aligned} \quad (3.2)$$

where

$$\begin{aligned} \alpha &= x_t^2 + y_t^2, \\ \beta &= x_s x_t + y_s y_t, \\ \gamma &= x_s^2 + y_s^2, \\ J &= x_s y_t - x_t y_s, \end{aligned} \quad (3.3)$$

which is known as the Winslow or Thompson-Thames-Mastin Generator. Because of the previously stated drawbacks to this approach, we avoid using an inversion strategy and pursue a different course, one which relies heavily on the notion of a reference grid.

Castillo, Steinberg and Roache [3] have introduced the concept of a reference mapping to provide additional flexibility in constructing useful functionals for variational grid-generation.

A rectangular reference grid has the form shown in Figure 6, where A and B are parameters which are considered to be problem dependent. This turns out to be a useful choice in practice, although there are other useful reference grids.

We place no restriction on the form that a reference grid may take; it must merely have the same dimensionality of the logical and physical grids. A reference grid is used to obtain a new

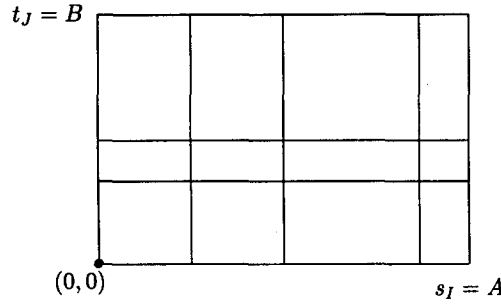


Figure 6. Rectangular reference grid.

functional for direct optimization by modifying the proportionality of the quantities which define a given functional.

For example, if we let $\{a_{ij}\}$ be the areas of the reference cells, then the area functional becomes

$$F_A(\mathbf{x}, \mathbf{y}) = \sum_{C_{ij}} \frac{A_{ij}^2}{a_{ij}^2}. \quad (3.4)$$

For length, we define reference lengths l_{ij}^h and l_{ij}^v , and let

$$F_L(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \sum_{E_{ij}} \frac{(L_{ij}^h)^2}{(l_{ij}^h)^2} + \frac{(L_{ij}^v)^2}{(l_{ij}^v)^2}. \quad (3.5)$$

It is also possible to define such a modified functional for orthogonality. However, special care must be taken; consider a rectangular reference grid, for which the reference quantities o_{ij} vanish.

One attractive feature of the use of a reference grid is that it allows for standardization of the logical grid: we may always take the logical/computational grid to be a unit grid with uniform mesh spacing in each direction (Figure 7). This, in turn, allows for standardization of the minimization algorithm.

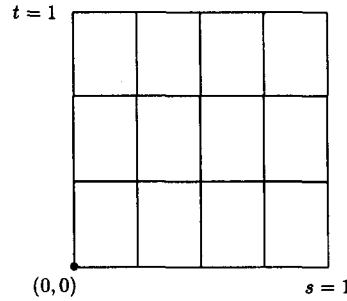


Figure 7. Uniform logical grid.

Now, when a rectangular reference grid with uniform spacing is used with a unit/uniform logical grid, the Hessian for the length functional is given by

$$H = \begin{bmatrix} H_{xx} & 0 \\ 0 & H_{yy} \end{bmatrix}, \quad (3.6)$$

where $H_{xx} = H_{yy}$ are matrices with the five-point stencil indicated in Figure 8.

Here $\mu = (A/B)^2$, and Σ is the summation of off-diagonal elements. This is a standard discretization for the equivalent problems of Figure 9. We make this point in order to emphasize

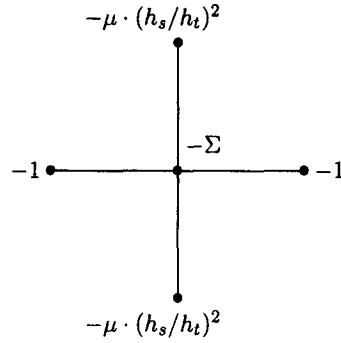


Figure 8. Stencil for lengths functional with a rectangular grid.

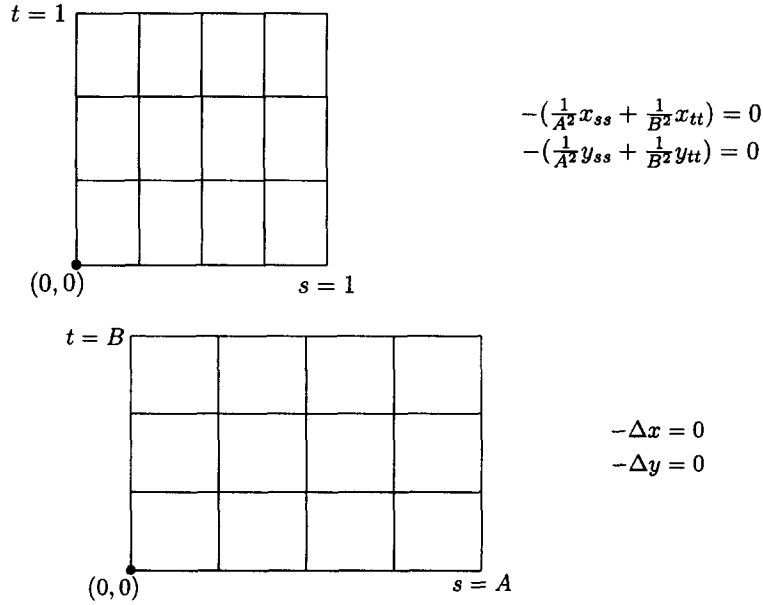


Figure 9. Equivalent elliptic systems under a rectangular grid.

the connection between the length functional used in conjunction with a reference grid and various elliptic grid-generators.

It turns out that the problem of folding associated with the length functional can in many instances be controlled by use of a reference grid. The following grids were obtained by a method described by the above equivalences and using $A = 1$. For a given geometry (i.e., physical domain), grids were computed for various values of the parameter B . The first example is for the S-geometry. Suppose we use the choice $B = 1$, which corresponds to the use of Laplace's equation as an elliptic generator. It also corresponds to the use of the trivial reference grid (i.e., a square grid with equal spacing), yielding the vanilla length functional. The resulting physical grid is severely folded, as shown in Figure A6. Now, the initial motivation behind the use of reference grids is to allow one to include information about the physical geometry in the grid functional. Suppose we include the aspect ratio of the sides of the physical domain by setting $B = 14$. Then we get the grid in Figure A7; this is a relatively robust procedure in that a suitable parameter may be chosen to produce an unfolded grid in many instances where the vanilla length functional gives a folded grid. In particular, most of the examples of the Rogue's Gallery in [4] can be treated this way.

Some questions remain, however. First, can the appropriate value for this parameter be chosen *a priori*, and second, is this strategy truly robust? In an attempt to answer the first part, we note

an interesting phenomenon. Suppose we increase the value of B in the previous example. Then one notices that when B is only moderately large the resulting grid becomes indistinguishable from the algebraic grid obtained from bilinear interpolation of the boundary conditions [5]. This is not an isolated phenomenon—it is pervasive over a wide selection of physical geometries. It seems reasonable, then, to incorporate information from the interpolation grid in the length functional in order to control folding. Before pursuing this, though, we address the question concerning robustness of the above strategy. This might be posed as follows: can we always find a B such that the mapping by the system

$$\begin{aligned} -\left(x_{ss} + \frac{1}{B^2}x_{tt}\right) &= 0, \\ -\left(y_{ss} + \frac{1}{B^2}y_{tt}\right) &= 0 \end{aligned} \quad (3.7)$$

of the unit square to an “arbitrary” domain is 1-to-1? The well-known answer is that one may do so by using a conformal mapping; for some B , the system

$$\begin{aligned} s_x &= B \cdot t_y, \\ s_y &= -B \cdot t_x \end{aligned} \quad (3.8)$$

maps a given domain to the unit square in such a way. Furthermore, the inverse mapping satisfies the above system of equations. (Notice that the mapping to the unit square is harmonic.) The problem of generating grids using variations on the Cauchy-Riemann equations, or grid-generation via quasi-conformal mappings, has been studied in [6,7]. Unfortunately, this mapping does not allow the freedom we require in assigning boundary values; for our purposes, the question really needs to be reposed in the context of specifying these values. Then the answer is negative. A counterexample is provided by the three-sided valley, or *fishtail*. This geometry/grid is pictured in Figure A8, for vanilla length ($B = 1$), and again in Figures A9 and A10 for $B = 1/2$ and 2, respectively. One finds that due to the symmetry of the problem, no value of B will produce a grid without folding at all of the concave sides. On the other hand, replacing the elliptic generator (3.7) with

$$\begin{aligned} -\left(x_{ss} + \frac{1}{B_1^2}x_{tt}\right) &= 0, \\ -\left(y_{ss} + \frac{1}{B_2^2}y_{tt}\right) &= 0 \end{aligned} \quad (3.9)$$

enables the generation of unfolded grids for this example. In particular, the choice $B_1 = 2$, $B_2 = 1/2$ yields the grid of Figure A11.

The use of distinct elliptic PDEs for mappings for x and y corresponds to the use of pairs of reference grids in direct optimization. As an example, recall the length functional under a single reference grid, (3.5). Its specific form for the case of a rectangular reference grid (and $A = 1$) is

$$F_L(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \sum_{E_{ij}} \frac{(x_{i+1,j} - x_{ij})^2 + (y_{i+1,j} - y_{ij})^2}{h_s^2} + \frac{(x_{i,j+1} - x_{ij})^2 + (y_{i,j+1} - y_{ij})^2}{B^2 h_t^2}. \quad (3.10)$$

When a pair of rectangular reference grids is used, this becomes

$$F_L(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \sum_{E_{ij}} \frac{(x_{i+1,j} - x_{ij})^2 + (y_{i+1,j} - y_{ij})^2}{h_s^2} + \frac{(x_{i,j+1} - x_{ij})^2}{B_1^2 h_t^2} + \frac{(y_{i,j+1} - y_{ij})^2}{B_2^2 h_t^2}. \quad (3.11)$$

A reformulation of the length functional which accounts for distinct reference grids could be given, although the notation becomes rather clumsy, and it does not seem natural. As an alternative, it may be preferable to think of weighting of the length functional as proceeding by various

strategies, one of which uses a single reference grid, and another of which uses *aspect weighting* as specified explicitly by (3.11), or more generally by

$$F_L(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \sum_{E_{ij}} \frac{(x_{i+1,j} - x_{ij})^2}{A_1^2 h_s^2} + \frac{(y_{i+1,j} - y_{ij})^2}{A_2^2 h_s^2} + \frac{(x_{i,j+1} - x_{ij})^2}{B_1^2 h_t^2} + \frac{(y_{i,j+1} - y_{ij})^2}{B_2^2 h_t^2}. \quad (3.12)$$

Returning to the previous example, we note once more that if we allow the reference parameters to converge toward limiting values (i.e., $B_1 \rightarrow \infty$ and $B_2 \rightarrow 0$), then the associated grids rapidly converge to the grid obtained by bilinear interpolation of the boundary values. However, for moderate values of the parameters, the resulting grid may have better properties than the interpolation grid. Compare the grid of Figure A11 with the interpolation grid for the **fishtail** pictured in Figure A12; the latter has severe cell compression inside the fins. Another example is provided by the **S**. In this case, bilinear interpolation propagates discontinuities at the boundary throughout the interior of the grid. The point to be made here is that the interpolation grid contains information which can be used to control folding. Some strategies for incorporating this information in the procedure for determining an unfolded grid are presented in the following section.

Before closing this section, we note that the strategy of choosing the coefficients in the system

$$\begin{aligned} -\left(\frac{1}{A^2}x_{ss} + \frac{1}{B^2}x_{tt}\right) &= 0, \\ -\left(\frac{1}{A^2}y_{ss} + \frac{1}{B^2}y_{tt}\right) &= 0 \end{aligned}$$

appropriately for nonconvex domains dates back to [8]. However, the use of distinct coefficients for x and y was not considered there.

4. INTERPOLATION AND THE LENGTH FUNCTIONAL

In this section, we consider two practical remedies to the problem of folding associated with the length functional. Both of these approaches use information supplied by the interpolation solution in order to modify the vanilla length functional. The first uses the notion of a reference grid. The second uses the interpolation solution to generate an appropriate right-hand side which is used in conjunction with the Hessian for the length functional to derive a linear system of equations. Although for clarity of exposition, we describe these procedures separately, they can be combined to good advantage, and we will eventually do so, obtaining a generalized length strategy.

As noted in the previous section, one approach to controlling folding is to incorporate the curvature of the physical domain into the definition of the coefficients which define a self-adjoint elliptic grid-generator. Unfortunately, it is a difficult task in general to determine *a priori* the right coefficients to control folding. But we have also seen there that an equivalence exists between certain elliptic grid-generators and the length functional under a reference grid. Suppose we exploit this connection by using the interpolation grid itself as the reference grid. This provides an automatic way to include information about the geometry, and to generate (discretized) coefficient functions. In the last section, we saw examples of folding problems associated with the vanilla length functional applied to the **ear**, **S**, and the **fishtail**. Figures A13–A15 present grids for these geometries generated via (3.5), i.e., the weighted length functional, where the weights are determined by using the interpolation grid as a reference grid. Attractive grids are generated in each case. An additional example is provided by the **swan**, which is considered to be a rather difficult geometry (see, for example, the discussion in [5]). The interpolation grid and the grid generated by length with interpolation weighting are shown in Figures A16 and A17.

One advantage of this approach is that the procedure is entirely automatic—once a boundary distribution has been determined, the user need not supply any information or interact with the

solution algorithm. Another advantage (over inverse methods, such as the Winslow generator) is that a system of uncoupled, linear equations may be solved to minimize the functional. Furthermore, this system corresponds to an elliptic partial differential system: a straightforward multigrid method may be used to compute a rapid solution. Indeed, this approach has been taken for all of the examples of this paper where a weighting of just the length functional is used. For further discussion of this topic, see Section 5. A final advantage of this approach is its robustness: it generates nonfolded grids for a wide variety of nonconvex geometries. This is not to say that this method is completely robust. In certain cases, the user will need to exert additional control over grid quality. The second approach we describe has more flexibility to allow this kind of control.

Suppose we write $\nabla F_L(\mathbf{z}) = 0$ as a linear system and incorporate the boundary conditions in the right-hand side:

$$H_L \mathbf{z}_L = \mathbf{f}_L. \quad (4.1)$$

Let $\mathbf{f}_I = H_L \mathbf{z}_I$, where $\mathbf{z}_I$ is the interpolation solution (restricted to interior nodes). We obtain a new mapping by taking a convex combination of length and interpolation right-hand sides:

$$H_L \mathbf{z}_{L-I} = (1-t) \cdot \mathbf{f}_L + t \cdot \mathbf{f}_I, \quad t \in [0, 1]. \quad (4.2)$$

For $t = 0$ this reproduces the length solution, and for $t = 1$ it reproduces interpolation. Thus, we get a family of grids which in some sense lie between length and interpolation. This is not entirely satisfactory, though, in that the grid is usually only ill-behaved in some localized region. It is more appropriate to weight the right-hand sides locally, using the length right-hand side alone where the grid is well-behaved. In order to incorporate local control, choose a set $S = \{(i, j)\}$ of nodes to weight, and let

$$\delta_{ij} = \begin{cases} 1, & \text{for } (i, j) \in S, \\ 0, & \text{for } (i, j) \notin S. \end{cases} \quad (4.3)$$

Then use instead

$$H_L \mathbf{z}_{L-I} = (1 - \delta_{ij}) \cdot \mathbf{f}_L + \delta_{ij} \cdot \mathbf{f}_I. \quad (4.4)$$

To determine the set S , define a logical distance function,

$$d((m, n), (i, j)) = |m - i| + |n - j|, \quad (4.5)$$

and a *target set* $T = \{(m, n)\}$. The latter set can usually be taken to consist of a small set of nodes, each of which is “centered” in a specific region where improvement of the grid is desired. Then let

$$S = \{(i, j) : d((m, n), (i, j)) \leq K, \text{ for all } (m, n) \in T\}, \quad (4.6)$$

for some (usually small) integer K . To demonstrate the use of (4.4), consider again the **fishtail**; recall the folding in Figure A8 associated with vanilla length. Taking $I = J = 17$, we generate a new grid using the target set, $\{((I+1)/2, 1), (I, (J+1)/2), ((I+1)/2, J)\}$, consisting of three boundary nodes, and $K = 1$ in (4.6). The result is shown in Figure A18.

In certain difficult cases, it turns out that neither of the strategies described above is adequate to supply sufficient control over grid quality. Consider the latter strategy of applying a mask to the interpolation right-hand side for the (backwards) **C**. We used a 17×33 grid with a target set of size 11 consisting of points surrounding the **C**’s horizontal slot, and $K = 5$. The result is shown in Figure A19. Although the grid is much smoother than the interpolation grid (see Figure A20), severe folding is evident despite appropriate masking of the interpolation right-hand side. Figure A21 shows the result of using a generalized length strategy for this geometry. In particular, the interpolation grid was used as a reference grid in weighting length. Then the Hessian for this functional was applied to the interpolation solution, yielding a new interpolation right-hand side. Finally, the same mask as above was applied to this right-hand side and the resulting linear system with Hessian as coefficient matrix and combination of right-hand sides was solved. Both smoothness and folding have been controlled by using a combined strategy.

5. A GENERALIZED LENGTH STRATEGY

In the last section, the concept of a generalized length functional was introduced. The advantage of using such a strategy, which incorporates various methods introduced earlier in this paper, was also demonstrated. We note that when this strategy is viewed as functional minimization, the functional is defined only implicitly: it is embodied in a constructive procedure for computing a grid, namely, solving a particular system of linear equations. In this section, we provide a formal definition for what we call a generalized length strategy, and discuss some of its properties. There are three basic components which define such a strategy: a generalized length functional (i.e., a functional derived from vanilla length by weighting), an associated interpolation right-hand side, and a mask for the latter.

A generalized length functional has the form:

$$F_L(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \sum_{E_{ij}} \frac{(x_{i+1,j} - x_{ij})^2}{A_{ij}^2} + \frac{(y_{i+1,j} - y_{ij})^2}{B_{ij}^2} + \frac{(x_{i,j+1} - x_{ij})^2}{C_{ij}^2} + \frac{(y_{i,j+1} - y_{ij})^2}{D_{ij}^2}, \quad (5.1)$$

which includes (3.5) as well as (3.12), that is, functionals based on reference grids and those based on aspect weighting (see below).

Once again, write $\nabla F_L(\mathbf{z}) = 0$ as a linear system in the matrix H_L and right-hand side $\mathbf{f}_L$ (corresponding to homogeneous boundary conditions), and let $\mathbf{u}_I$ be the solution obtained by interpolating a specified boundary relation on the physical domain (as with transfinite interpolation or a similar procedure). The interpolation right-hand side is obtained by applying the Hessian to the interpolation solution:

$$\mathbf{f}_I = H_L \mathbf{u}_I. \quad (5.2)$$

A mask for the right-hand side is obtained by generating a Kronecker- δ function as described in the previous section (using (4.3), (4.5), and (4.6)). The generalized length strategy consists, then, of solving the system of equations

$$H_L \mathbf{z} = (1 - \delta_{ij}) \cdot \mathbf{f}_L + \delta_{ij} \cdot \mathbf{f}_I, \quad (5.3)$$

to obtain the mapping represented by $\mathbf{z}$.

We note that any of the previously expounded strategies based on the length functional (as well as the interpolation grid itself) may be reproduced by choosing parameters appropriately in generalized length. In particular, the vanilla length functional is reproduced by taking $A_{ij} = B_{ij} = C_{ij} = D_{ij} = 1$. Weighting length by a reference grid is done by setting $A_{ij} = B_{ij} = l_{ij}^h$ and $C_{ij} = D_{ij} = l_{ij}^v$ in (3.4). One obtains an aspect-weighted functional similarly via (3.12) by setting $A_{ij} = A_1 h_s$, $B_{ij} = A_2 h_s$, $C_{ij} = B_1 h_t$, $D_{ij} = B_2 h_t$. Grids which correspond to minimization of one of these functionals are obtained by using the appropriate functional to define H_L and setting $\delta_{ij} \equiv 0$. The second strategy of Section 4 (using vanilla length along with an interpolated right-hand side) is reproduced by taking the generalized length functional to be vanilla length and defining appropriate target sets, etc. Finally, the interpolation grid is reproduced by using any generalized length functional which generates a nonsingular H_L and taking $\delta_{ij} \equiv 1$.

We can always view (5.2) as a discretization of an uncoupled, elliptic, self-adjoint system of PDEs on the unit square and using a uniform mesh. Therefore, we may use a simple multigrid algorithm to obtain a rapid solution of (5.2). All of the examples in this paper which are based on length were computed using multigrid based on Gauss-Seidel smoothing. In certain cases, we have added flavors to the solution procedure by considering alternative smoothers, such as variations on red-black and weighted block-Jacobi smoothing. Such alternatives may be useful in certain situations. Consider the case of an aspect-weighted functional with $A_1/B_1 \gg 1$, for example. This corresponds to a highly anisotropic diffusion problem. Therefore, some advantage would be obtained in this case by basing multigrid on appropriate line smoothing. However, a potential

user need not be proficient in multigrid to solve the required linear systems: a user-transparent black box multigrid solver [9], which provides some smoothing options, can be used. Conjugate gradients preconditioned by MILU factorization combined with Schur complement reduction [10] is also a viable and robust method which can be tuned to deal with anisotropy [11].

Finally, we note that the generalized length strategy extends formally to three dimensions by the trivial inclusion of another coordinate. A thorough treatment of the topic of generalized length can be found in [12].

6. DIRECT OPTIMIZATION AND THE ALIGNMENT PROBLEM

The direct optimization generator for the problem of aligning a grid with a given vector field was introduced in [13]. We give a brief description of this generator here, and note some practical procedures which may be used to advantage in its implementation. Let $\mathbf{v} = (a(x, y), b(x, y))^T$ be a given vector field. It turns out that as a matter of convenience, it is useful to normalize its components: let

$$\hat{a} = \frac{a}{\|\mathbf{v}\|}, \quad \hat{b} = \frac{b}{\|\mathbf{v}\|}. \quad (6.1)$$

Like the hyperbolic generator of [6], the variational generator considered in [13] is based on the transformation defined implicitly by the system

$$\begin{aligned} x_s &= \hat{a}, & x_t &= -\hat{b}, \\ y_s &= \hat{b}, & y_t &= \hat{a}. \end{aligned} \quad (6.2)$$

(Note, the implementation of the hyperbolic generator in [6] uses a slightly different scaling of the vector field.) The hyperbolic generator attempts to integrate these equations directly, while the direct optimization functional can be viewed as least squares minimization applied to a finite difference version of this system. Another interpretation of the latter is as the elliptic generator corresponding to the system

$$\begin{aligned} -(x_{ss} + x_{tt}) &= \hat{b}_t - \hat{a}_s, \\ -(y_{ss} + y_{tt}) &= -(\hat{a}_t + \hat{b}_s). \end{aligned} \quad (6.3)$$

A similar strategy is used by Knupp in [4,14] for a comparison of that method with the one we propose. Advantages of the direct optimization approach include, again, avoidance of various inversions required to ensure unicity of the mapping. Another reason for our preferring direct optimization is that the resulting functional is compatible with the length functional as described earlier. This means that the various strategies designed to enhance control in lengths can also be used for alignment grids. An example of this is given below.

When (6.3) is discretized by standard means, one obtains the (nonlinear) system

$$H_L \mathbf{z} = \mathbf{f}(\mathbf{z}). \quad (6.4)$$

Notice that the nonlinearity is restricted to the right-hand side; the coefficient matrix is the Hessian associated with the vanilla length functional.

In [9,13], the phenomenon of “locking” associated with alignment of a grid through variational methods is considered. In the first article, a procedure of alternating variational solves with the updating of boundary values was introduced to avoid locking. The procedure of “iterating boundaries” was refined by the present authors in the latter reference. We alternate the solution of (6.4) (which accepts a current set of interior grid values and a current set of boundary values as input—and returns an updated set of interior values) with projection of near-boundary grid nodes in the characteristic directions associated with the vector field (the cycle can be initialized with an equidistribution of boundary values and an interpolation solution for the interior values).

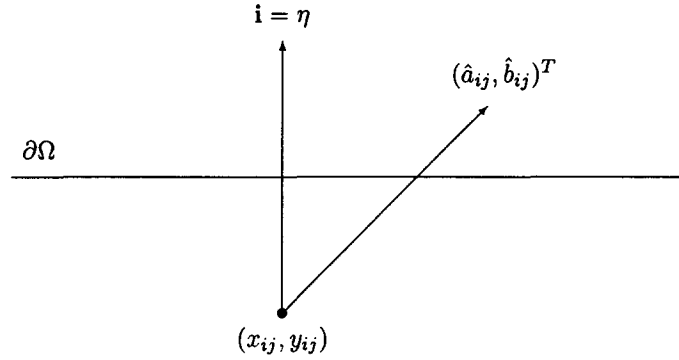


Figure 10. Directions associated with a near-boundary node.

The refined procedure allows this projection to occur in convex combinations of characteristic and normal directions. Consider the near-boundary node shown in Figure 10, which has a connection to the physical boundary in the (logical) s -direction.

Let $\mathbf{i}$ be the unit normal vector and $\mathbf{v} = (\hat{a}, \hat{b})^\top$. Then projection to the boundary is done in the direction

$$\mathbf{w} \equiv \frac{\hat{\mathbf{w}}}{\|\hat{\mathbf{w}}\|}, \quad \hat{\mathbf{w}} = \alpha \mathbf{i} + (1 - \alpha) \mathbf{v}, \quad (6.5)$$

for some value of $\alpha \in [0, 1]$.

Newton's method for (6.4) has the form

$$\mathbf{z}_{n+1} = \mathbf{z}_n - J(\mathbf{z}_n)^{-1}(H_L \mathbf{z}_n - \mathbf{f}(\mathbf{z}_n)), \quad (6.6)$$

where $J(\cdot)$ is the Jacobian for $F(\mathbf{z}) = H_L \mathbf{z} - \mathbf{f}(\mathbf{z})$. Notice that

$$J = \begin{bmatrix} J_{11} & J_{12} \\ J_{12}^\top & J_{22} \end{bmatrix}, \quad (6.7)$$

where J_{11} and J_{22} are five-point nearest neighbor stencils and J_{12} is a similar four-point stencil. For efficiency in the required inversions, we considered truncated Newton's methods,

$$\mathbf{z}_{n+1} = \mathbf{z}_n - \hat{J}(\mathbf{z}_n)^{-1}(A\mathbf{z}_n - \mathbf{f}(\mathbf{z}_n)) \quad (6.8)$$

which use

$$\hat{J} = \begin{bmatrix} J_{11} & D \\ E & J_{22} \end{bmatrix}, \quad (6.9)$$

in [13]. More recently, however, we have discovered that it may be more efficient to simply treat (6.4) as a linear system as a result of updating its right-hand side with the current grid (obtained from the previous solution of (6.4) along with a boundary update). The real convergence problem is that, typically, many iterations of the solution/update pair are required. One way we have found to reduce this number effectively is to use a more accurate initial guess for the boundary values. Such an initial guess can be obtained by nested iteration of the algorithm on a sequence of coarsened grids; interpolation of the final solution on each coarse grid to the next finer grid provides an initial fine guess. On the coarsest grid, (transfinite) interpolation is used to start things up; then multigrid is used to solve (6.4), which is alternated with boundary updates, on each level. Since we treat the right-hand side of (6.4) as constant for each solve, multigrid is again being applied to what is essentially an uncoupled, linear, self-adjoint, elliptic system, ensuring rapid solves.

We present an example of use of the method described above for the alignment problem represented by the flow field of Figure A22. A more detailed treatment of this example can be found

in [13]. Figure A23 shows the solution of this problem using projection of boundary values in purely characteristic directions ($\alpha = 0$ in (6.5)). Although good alignment results without auxiliary control, severe disparity in node spacing occurs at the boundaries. To remedy this situation, we applied (6.5) with $\alpha = 0.9$ on the horizontal boundaries. Unfortunately, the resulting grid (not shown here, but see [13]) suffers from severe compression of cells near the south boundary. However, this malady can be treated by using a masking of the interpolated right-hand side as described above in Sections 4 and 5. The grid obtained using a combination of these auxiliary techniques is shown in Figure A24.

We note that there are at present a variety of variational generators for the alignment problem: see [9,14,15]. These lack, however, the kind of precise boundary control we have implemented for the functional described here. Each of these methods can be implemented, though, as a procedure which alternates solving a boundary value problem and iterating boundaries (indeed, the idea originated in [9]). Therefore, each method can also be used in conjunction with the method described here for auxiliary grid control through manipulation of boundary values. On the other hand, one advantage of the functional described here is its consistency with the lengths functional. This means that (as demonstrated by the above example) procedures which we have developed to enhance the quality of grids generated by length can also be used for alignment grids.

7. CONSTRAINED MINIMIZATION

The minimization of the direct optimization grid generation functionals can be cast as a constrained minimization problem. It has been shown that it is possible to have grids where the Jacobian of the transformation (the area of the cells) is everywhere positive (i.e., the cell areas are all positive) but the grid is folded [16]. Alternatively, by solving a minimization problem where the constraint is that the area of each triangle (i.e., each half-cell, see (3.7)) be positive, we guarantee a nonfolded grid [17]. For each physical cell C_{ij} , as in Figure 4, the area of C_{ij} is the sum, see (3.7),

$$A_{ij} = A(Q, R, S) + A(Q, S, T)$$

and the constrained minimization problem is: find $\mathbf{x}, \mathbf{y}$ that minimize

$$F_L(\mathbf{x}, \mathbf{y}) = \sum_{E_{ij}} (L_{ij}^h)^2 + (L_{ij}^v)^2 \quad (7.1)$$

subject to the constraints that both $A(Q, R, S)$ and $A(Q, S, T)$ are greater than zero. This approach should be used only in some extreme cases because it is more expensive due to the constraints. In addition, the grids obtained in our experiments were not substantially better, judging by the plots, than the ones produced by a good reference grid.

8. ADAPTION

Solution adaptive methods enhance the accuracy of the numerical simulation, so there has been extensive work applying them to grid generation. Some references are [11,18–24]. There are different types of adaption. We choose to use feature adaption, which consists of keeping the number of grid points constant and moving the points to the place where they are needed, namely, where there is rapid change in the solution. This strategy is called feature adaption because it concentrates on a particular feature like pressure or gradient to define the variability of the solution that it attempts to capture. Note that this approach need not be limited to a single feature. There are two basic ways to perform feature adaption, both of which are based on an equidistribution principle. The latter is intended to equidistribute the error over the whole domain. Adaption is done by way of weights that modify the grid generation functionals. The approach we favor, based on experience, is adaption via the reference grid. We use computed weights to modify the reference grid so that information about the features of the solution to

which we want to adapt are passed to the grid via the reference grid. Here is the adaption procedure in detail.

- Step 1: Generate optimal initial grid.
- Step 2: Compute flow solution on initial grid.
- Step 3: Generate solution adapted grid.
- Step 4: Compute flow solution on adapted grid.
- Step 5: Evaluate results, go to Step 3 if required.

The present adaptive technique is based on redistributing the existing grid points in order to resolve regions of high solution gradients. The grid point redistribution is governed by the principle of equidistribution wherein the product of the grid spacing and a specified weight function is constant over the field. The adaptive weight function is defined such that the grid points will adapt to the gradient of any specified flow quantity. It is carried out as a series of multiple one-dimensional adaptations along parameterized curvilinear coordinate lines. The parameterization is defined by transforming the 2-D physical space curve (e.g., s - or t -coordinate line) into a 1-D curve, based on normalized arc length. Using this approach, the grid points are constrained to move only along one family of fixed coordinate lines during each adaptive stage. The adapted grid point distribution along a parameterized coordinate line is obtained by minimizing a one-dimensional adaptive length functional. The adaptive length functional is formulated based on the principle of equidistribution. The adapted parameterized arc length distributions are used to define an adapted reference grid. Reference length and area weights based on the adapted reference grid are used to drive the grid point redistribution in 2-D. The final 2-D adapted grid is obtained by minimizing the length and area functionals, utilizing the adapted reference grids.

8.1. Solution Adaptive Length Functional 1-D

As stated above, the grid point redistribution is governed by the principle of equidistribution wherein a grid point distribution is defined such that the product of the grid spacing and a positive weight function (w_i) is constant over the field:

$$\Delta x_i w_i = \text{constant}, \quad (8.1)$$

where Δx_i is the grid spacing, $\Delta x_i = x_{i+1} - x_i$, and the weight function w_i is defined as some measure of the solution error or solution variation over the grid interval. This condition requires that the grid spacing be small where the weight function is large and vice versa. Based on the principle of equidistribution (8.1), a one-dimensional adaptive length functional is defined as the sum of the squares of the product of the grid segment and the average weight function over that segment:

$$F_{\text{ADP}} = \sum_{i=1}^{m-1} (l_i \bar{w}_i)^2, \quad (8.2)$$

where l_i is the grid length segment given by $l_i = (x_{i+1} - x_i)$ and $\bar{w}_i$ is the average weight functional over that interval given by $\bar{w}_i = (w_i + w_{i+1})/2$. The adapted grid point distribution is obtained by minimizing (8.2) using the conjugate gradient method.

8.2. Adaptive Weight Formulation

As stated previously, the accuracy of discrete solutions strongly depends on the local grid spacing and the resolution of solution gradients. Grid density, or cell size, is very important in affecting solution accuracy. For a fixed order of spatial discretization, the truncation error is a direct function of cell size [11,18,21]. Although the truncation error cannot be estimated accurately, in general, a finer grid produces less truncation error. Grid density is also important

in the resolution of complex flow phenomena such as shock waves, boundary layers, free shear layers, flow separation, and recirculation zones. The “accuracy” of the discrete solution is not only measured by the magnitude of the truncation error, but is also measured by how accurately the numerical solution captures the complicated flow phenomena present in the physical flowfield. Flowfield phenomena such as shock waves, boundary layers, and free shear layers are characterized by regions of rapidly changing flow variables. If the weight function is defined as a measure of the solution variation, the simplest choice is to define the weight as a function of the solution gradient ($\frac{\partial u}{\partial x}$), as in the form:

$$w = \sqrt{1 + \left(\frac{\partial u}{\partial x}\right)^2}. \quad (8.3)$$

A weight function of this form concentrates grid points where the solution gradient is large and produces uniform grid spacing where the solution curve is flat. This characteristic is desired in order to ensure that regions of little or no solution variation are not completely devoid of grid points. Adapting the grid point distribution using a weight function given by (8.3) is referred to as “flowfield feature adaption”, since the solution gradient is a direct measurement of various flowfield features such as shock waves, boundary layers, free shear layers, etc. The form of the weight function given by (8.3) is derived by defining a grid point distribution which produces equal increments of arc length (ds) along the solution curve $u(x)$ as shown in Figure A25:

$$ds = \sqrt{dx^2 + du^2} = dx \sqrt{1 + \left(\frac{du}{dx}\right)^2} = \text{constant}.$$

To provide for more concentration of points in the high gradient regions, a scaling factor is applied to the gradient term:

$$w = \sqrt{1 + \beta \left(\frac{\partial u}{\partial x}\right)^2}, \quad (8.4)$$

where β is a positive parameter to be specified. The larger the value of β , the stronger the concentration of grid points in high gradient regions and the wider the spacing where there is little or no change in the solution. The solution gradient used to define the adaptive weight function can be based on any of the scalar flow quantities such as pressure, Mach number, density, temperature, etc. In many studies on solution adaptive gridding [11,14,21], the weight function is defined by a linear combination of the solution gradient and curvature, similar to the form given by:

$$w^2 = 1 + \beta \left(\frac{\partial u}{\partial x}\right)^2 + \alpha \left(\frac{\partial^2 u}{\partial x^2}\right)^2. \quad (8.5)$$

The addition of the curvature term makes the weight function large, and hence concentrates grid points at solution extrema. In the present study, the addition of a solution curvature term did not produce any desirable properties in the resulting weight distribution. The weights given by (8.5) actually produced oscillations in the weight distribution that were unacceptable.

8.3. Solution Adaptive Grid Procedure

This procedure is summarized below for an adaption along the s -coordinate line. Given the geometry and solution on the s -coordinate line which defines a surface at the nodes of the initial grid $(x(s, 1), y(s, 1), u(s, 1))$.

- (1) Transform the 2-D physical space curve (s -coordinate line) into a 1-D parameterized curve based on normalized arc length (S):

$$s(x, y) \rightarrow s(S).$$

- (2) Select the flow quantity (u) whose gradient will be used to define the adaptive weights (i.e., Mach number, pressure, temperature, velocity, etc).
- (3) Smooth the solution curve $u(s)$ as necessary using the formula:

$$u_i^{n+1} = \frac{1}{4} (u_{i-1}^n + 2u_i^n + u_{i+1}^n).$$

- (4) Spline-fit the smoothed, parameterized solution curve $u(S)$ using cubic spline interpolation. Evaluate $(\frac{\partial u}{\partial S})_i$ at each node by taking the derivative of the piece-wise cubic polynomial.
- (5) Specify the gradient factor β and compute the adaptive weights using:

$$w_i = \sqrt{1 + \beta \left(\frac{\partial u}{\partial S} \right)_i^2}.$$

- (6) Smooth the resulting adaptive weights curve $w(S)$ as necessary using the formula:

$$w_i^{n+1} = \frac{1}{4} (w_{i-1}^n + 2w_i^n + w_{i+1}^n).$$

- (7) Spline-fit the smoothed adaptive weights curve $w(S)$ using cubic spline interpolation. This is required to determine w_i and $(\frac{\partial w}{\partial S})_i$ during the conjugate gradient minimization process.
- (8) Obtain the adapted grid point distribution by minimizing the adaptive length functional F_{ADP} using the conjugate gradient method:

$$\text{Minimize } F_{\text{ADP}} = \sum_{i=1}^{m-1} (l_i \bar{w}_i)^2,$$

where l_i is the parameterized arc length segment given by $l_i = (S_{i+1} - S_i)$ and $\bar{w}_i$ is the average weight functional over that interval given by $\bar{w}_i = (w_{i+1} + w_i)/2$.

- (9) Transform the adapted parameterized arc length distribution S^* to physical space coordinates (x, y) .

As stated previously, the two-dimensional adapted grid is produced by minimizing the 2-D DV length and area functionals utilizing the adapted reference grid. The adapted reference grid is obtained by performing a series of 1-D adaptations along a family of fixed, parameterized coordinate lines. The 1-D adaptive formulation and procedures outlined in the previous sections are utilized to perform a series of multiple 1-D adaptations in order to produce a 2-D adapted reference grid.

This approach maintains the original structure of the DV functionals and the convergence properties of the conjugate gradient solver. In addition, the present approach does not require CPU intensive multidimensional interpolation during the minimization process.

The solution adaptive grid procedure implemented in the present algorithm has been shown to provide excellent control over grid line clustering in regions of high flow (solution) gradients while maintaining a smoothly varying grid line distribution over the field. The present formulation provides for a high degree of grid line clustering in high gradient regions without other regions becoming devoid of points. The Discrete Variational grid generation method coupled with the solution adaptive grid procedure produce a powerful tool for generating solution adaptive grids. Results for transonic flow past an airfoil are presented in order to demonstrate the improvement obtainable with this solution adaptive grid procedure [12]. For example, the flow solution on the initial grid (see Figure A25) calculates a shock wave which has a thickness of approximately 10% of the chord, when in reality the shock wave is infinitesimally thin. The discrete solution in this region does not accurately predict the thickness of the shock and therefore its exact location on the airfoil surface. Figure A25 presents the initial optimal grid with the corresponding reference grid and the Mach contours for the flow field solution. The second column shows the adapted

grid, adapted reference grid, and the Mach contours after three adaptations. Note that the initial reference grid was constructed to place more points at the leading edge and the trailing edge of the airfoil because it is known that more resolution is needed at these parts. However, there is no prior knowledge of where the shock is going to appear. The improvement in shock wave resolution on the surface of the airfoil in transonic flow demonstrates the improvement obtainable with this solution adaptive procedure.

9. CONCLUSIONS

Direct optimization is a powerful form of variational grid generation. We have used it to define a generalized length strategy which gives a robust and efficient means of computing high quality planar grids even for nonconvex geometries. In particular, standard fast multigrid solvers can be used for the resulting system of linear equations. We have also used direct optimization to design an efficient alignment functional, which is comparable to the length functional. This means that techniques which comprise the generalized length strategy can also be applied in the alignment case to control grid quality. Direct optimization in conjunction with the use of reference grids also provides a framework for performing efficient computation of feature adaptive grids.

REFERENCES

1. J.E. Castillo, A discrete variational grid generation method, *SIAM J. Sci. Stat. Comput.* **12**, 454–468 (1991).
2. J.E. Castillo and P. Barrera-Sanchez, A large scale nonlinear optimization problem arising from numerical grid generation, (in preparation).
3. J.L. Steger and R.L. Sorenson, Automatic mesh-point clustering near a boundary in grid generation with elliptic partial differential equations, *Journal of Computational Physics* **33**, 405–410 (1979).
4. P.M. Knupp and S. Steinberg, *The Fundamentals of Grid Generation*, CRC Press, Boca Raton, FL, (1993).
5. A.E. Giannakopoulos and A.J. Engel, Directional control in grid generation, *J. Comp. Phys.* **74**, 422–439 (1988).
6. C.W. Mastin and J.F. Thompson, Discrete quasiconformal mappings, *Z. Angew. Math. Phys.* **29**, 1–11 (1978).
7. C.W. Mastin and J.F. Thompson, Quasiconformal mappings and grid generation, *SIAM J. Sci. Stat. Comput.* **5**, 305–310 (1984).
8. W.D. Barfield, An optimal mesh generator for Lagrangian hydrodynamic calculations in two space dimensions, *J. Comp. Phys.* **6**, 417–429 (1970).
9. J.E. Dendy, Jr., Black box multigrid, *J. Comp. Phys.* **48**, 366–386 (1982).
10. O. Axelsson and I. Gustafsson, On the use of preconditioned conjugate gradient methods for red-black ordered five point difference schemes, *J. Comput. Phys.* **35**, 284–289 (1980).
11. H.J. Kim and J. Thompson, Three dimensional adaptive grid generation on a composite block grid, AIAA Paper 88-0311, *AIAA 26th Aerospace Sciences Meeting*, Reno, NV, January 1988.
12. J.E. Castillo and J.S. Otto, A generalized length strategy for direct optimization in planar grid generation, *Mathematics and Computers in Simulation* **44** (5), 441–456 (1997).
13. J.E. Castillo and J.S. Otto, On the transformation to an orthogonal coordinate system aligned with a vector field, Part Two: Variational grid-generation, (in preparation).
14. P.M. Knupp, Mesh generation using vector fields, *J. Comput. Phys.* **119**, 142–148 (1995).
15. J.U. Brackbill, An adaptive grid with directional control, *J. Comp. Phys.* **108**, 38–50 (1993).
16. P.J. Roache and S. Steinberg, A new approach to grid generation using a variational formulation, AIAA Paper 85-1527, *AIAA 7th Computational Fluid Dynamics Conference*, Cincinnati, OH, July 1985.
17. A.D. Harvey, S. Acharya, S.L. Lawrence and S. Cheung, A solution adaptive grid procedure for an upwind parabolized flow solver, AIAA Paper 90-1567, *AIAA 21st Fluid Dynamics, Plasma Dynamics, and Lasers Conference*, Seattle, WA, June 1990.
18. J.S. Abolhassani and R.E. Smith, Multiple-block grid adaption for an airplane geometry, *Proc. 2nd Int. Conf. in Numerical Grid Generation in Computational Fluid Mechanics*, Miami Beach, FL, December 1988, pp. 815–823.
19. D.A. Anderson, Constructing adaptive grids with Poisson grid generators, *Proc. 1st Int. Conf. in Numerical Grid Generation in Computational Fluid Dynamics*, July 1986, pp. 125–136.
20. R.A. Benson and D.S. McRae, A three dimensional dynamic solution-adaptive mesh algorithm, AIAA Paper 90-1566, *AIAA 21st Fluid Dynamics, Plasma Dynamics, and Lasers Conference*, Seattle, WA, June 1990.
21. W.J. Gordon and L.C. Thiel, Transfinite mappings and their application to grid generation, In *Numerical Grid Generation*, (Edited by J.F. Thompson), North-Holland, New York, (1982).

22. K.D. Lee, J.M. Loellbach and T.R. Pierce, Solution adaptive grid generation using a parametric mapping, *Proc. 2nd Int. Conf. in Numerical Grid Generation in Computational Fluid Mechanics*, Miami Beach, FL, December 1988, pp. 455-464.
23. K.D. Lee, J.M. Loellbach and M.S. Kim, Adaption of structured grids for improved Navier-Stokes solutions, AIAA Paper 90-0125, *AIAA 28th Aerospace Sciences Meeting*, Reno, NV, January 1990.
24. W. Shyy, An adaptive grid method for Navier-Stokes flow computation, *Applied Mathematics and Computation* **21**, 201-219 (1987).
25. J.E. Castillo and E. Pedersen, Solution adaptive direct variational grids for fluid flow computations, *Journal of Computational and Applied Mathematics* (to appear).
26. J.E. Castillo, P.J. Roache and S. Steinberg, Mathematical aspects of variational grid generation II, *J. Comp. Appl. Math.* **20**, 127-135 (1987).
27. K.L. Kipp, J.S. Otto and T.F. Russel, D4Z—A new renumbering for iterative solution of ground-water flow and solute-transport equations, In *Proceedings of the IX International Conference on Computational Methods in Water Resources*, Denver, CO, June 9-12, 1992.
28. T.A. Manteuffel and J.S. Otto, On the transformation to an orthogonal coordinate system aligned with a vector field, Part One: Hyperbolic grid-generation, Manuscript.
29. S. Steinberg and P.J. Roache, Variational grid generation, *Num. Meth. for P.D.E.s* **2**, 71-96 (1986).
30. P.D. Thomas and J.F. Middlecoff, Direct control of the grid point distribution in meshes generated by elliptic equations, *AIAA Journal* **18** (6), 652-656 (1980).
31. V. Kazakov, Private communication.

APPENDIX

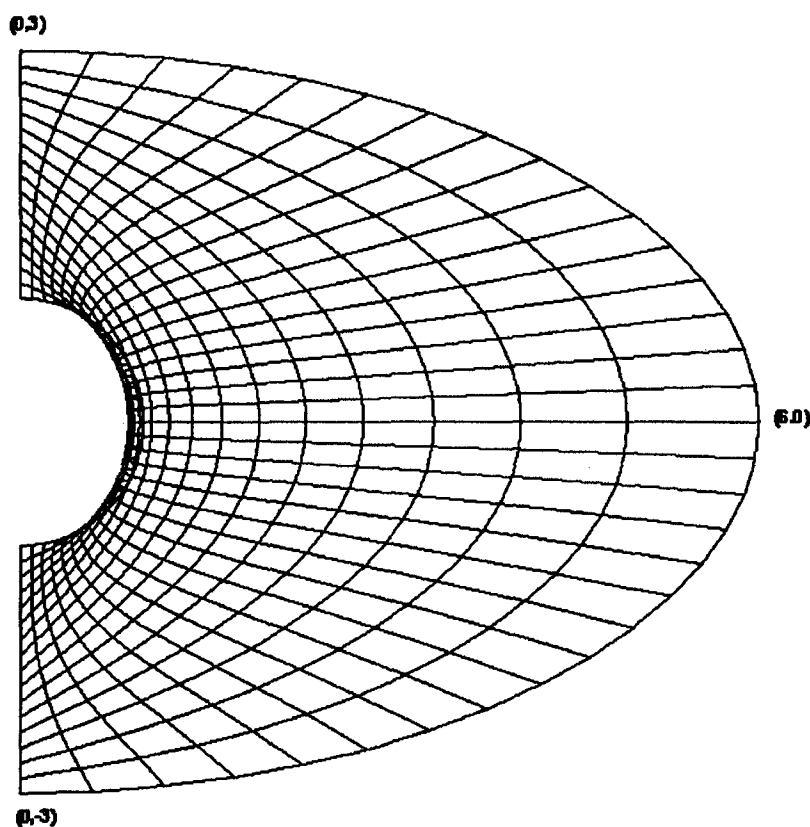


Figure A1. Vanilla lengths grid for the ear.

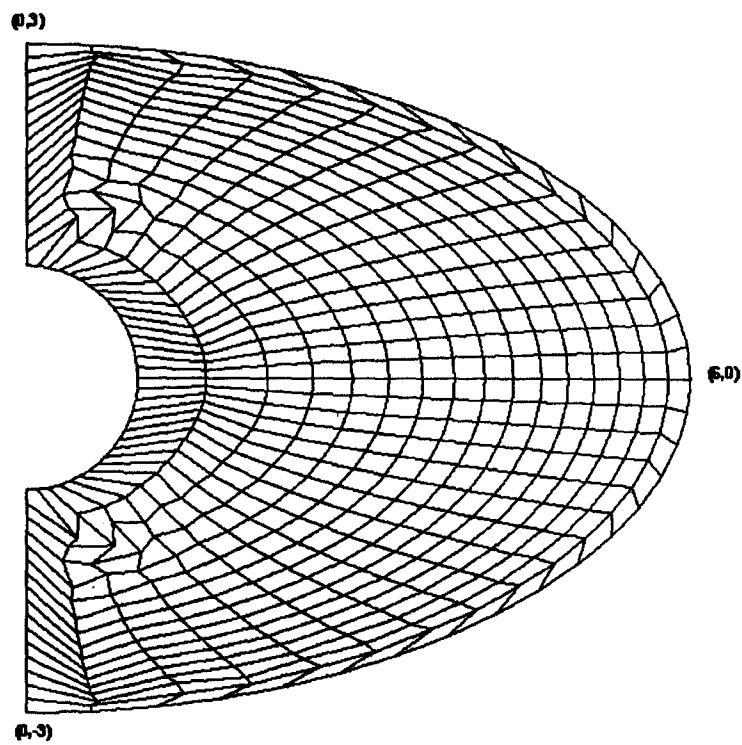


Figure A2. Vanilla area grid for the ear.

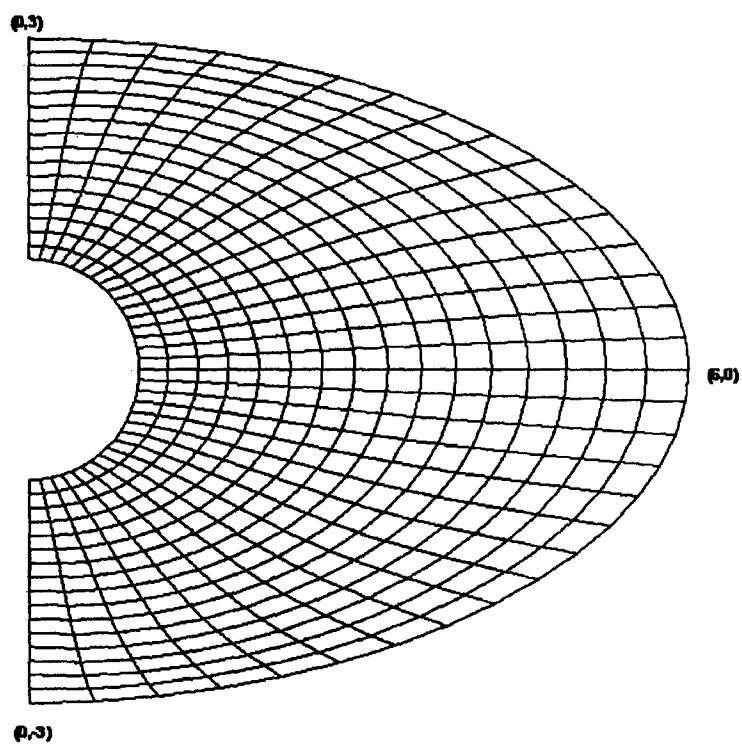


Figure A3. Combination of length and area for the ear.

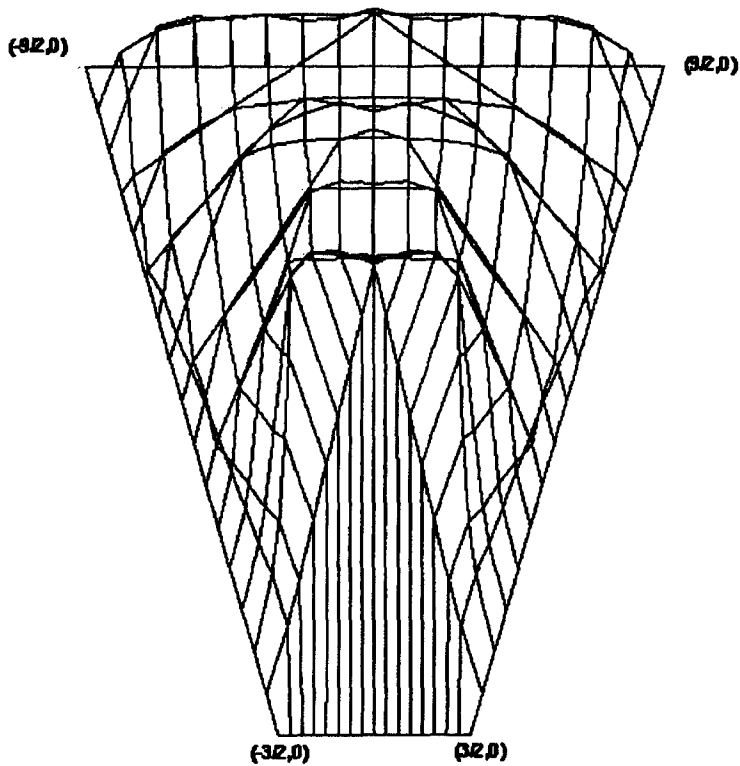


Figure A4. Orthogonality grid for the **trapezoid**.

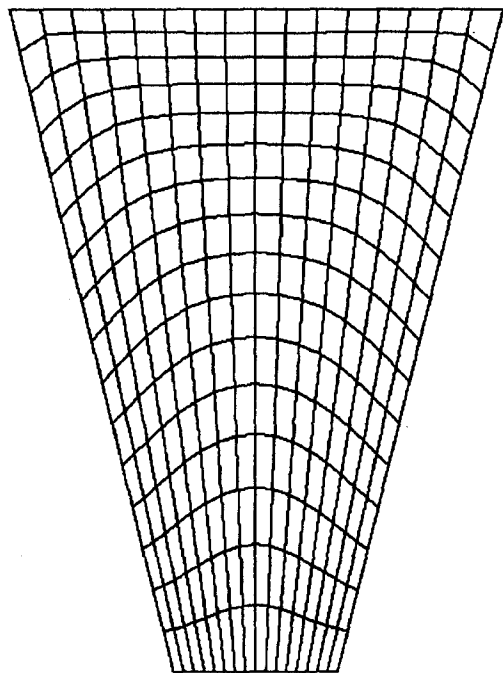


Figure A5. Combination of length and orthogonality for the **trapezoid**.

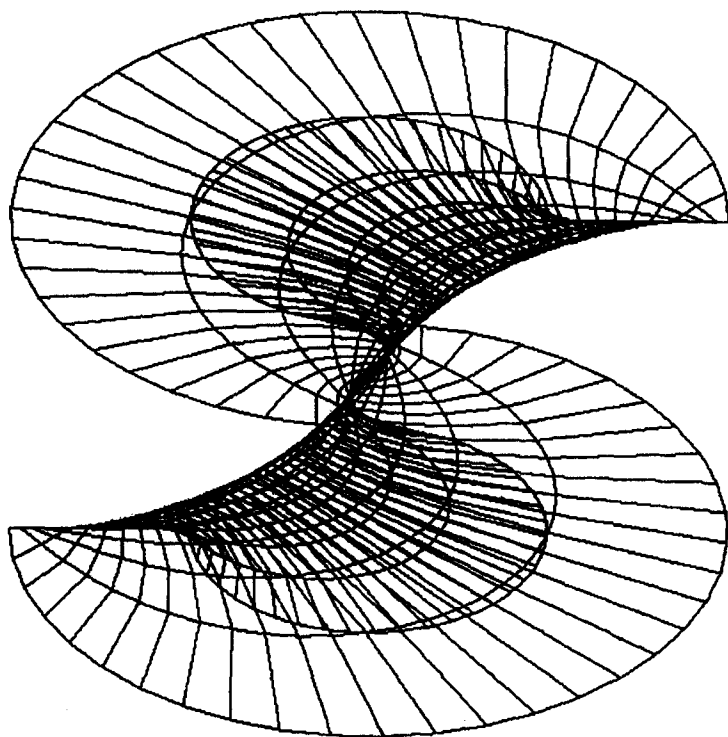


Figure A6. Vanilla length for the S.

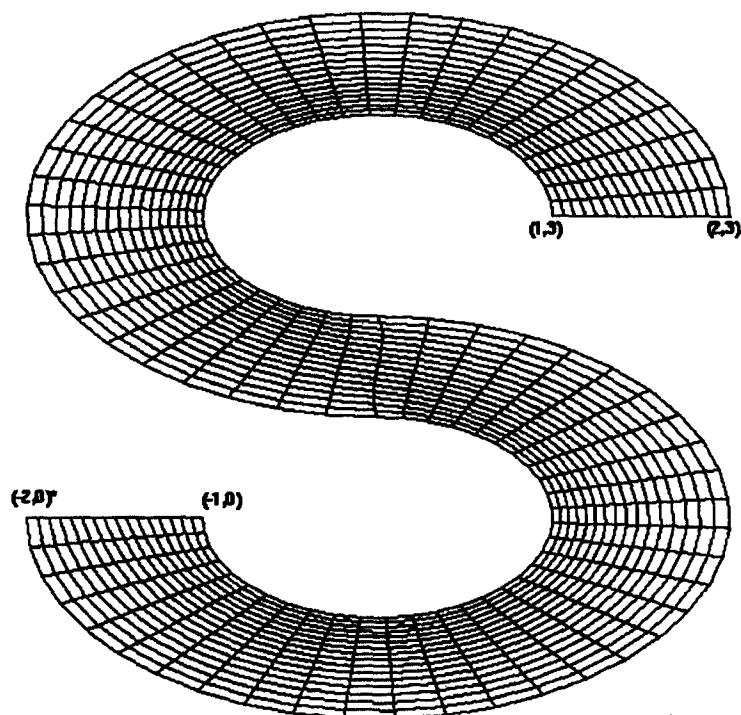


Figure A7. Length with rectangular reference grid for the S.

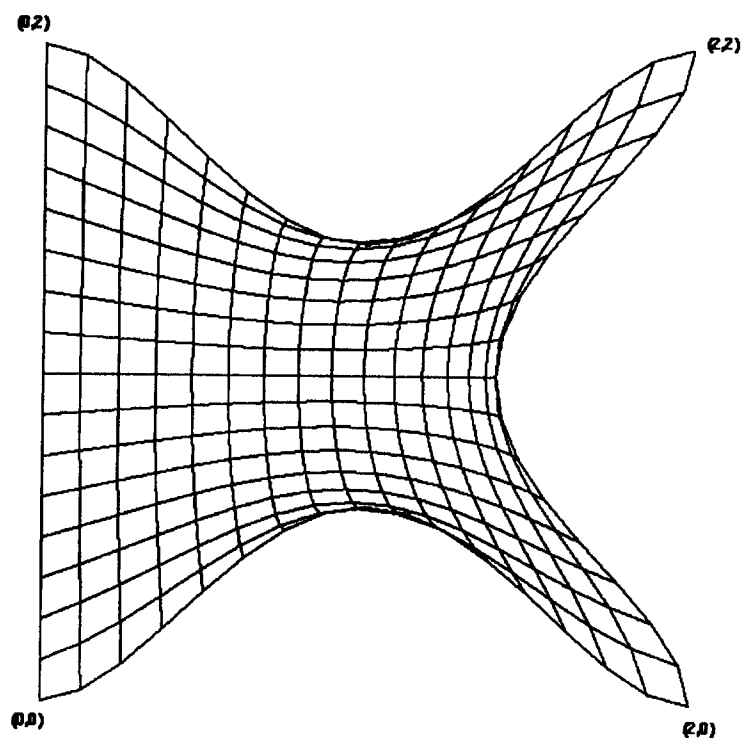
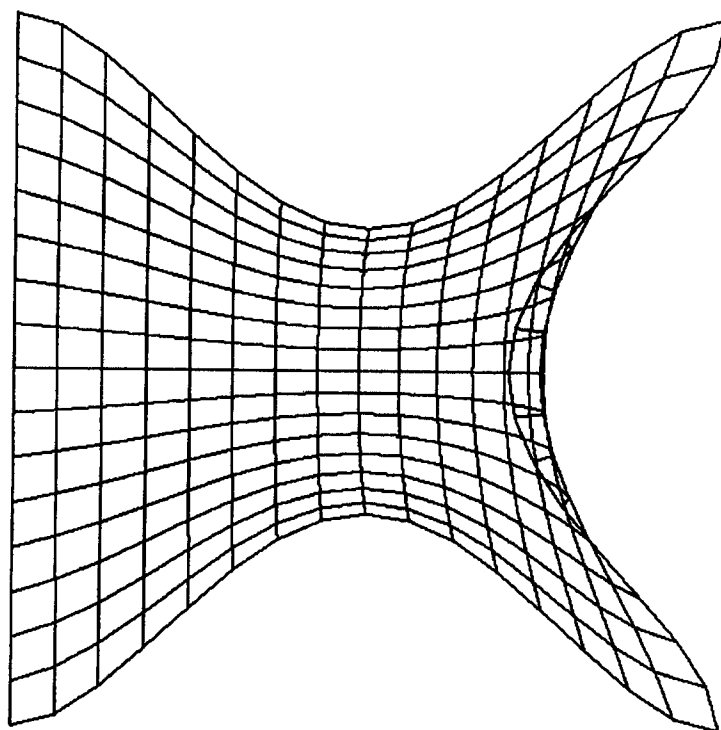


Figure A8. Vanilla length for the fishtail.

Figure A9. Rectangular reference grid ($A = 1, B = 1/2$) for the fishtail.

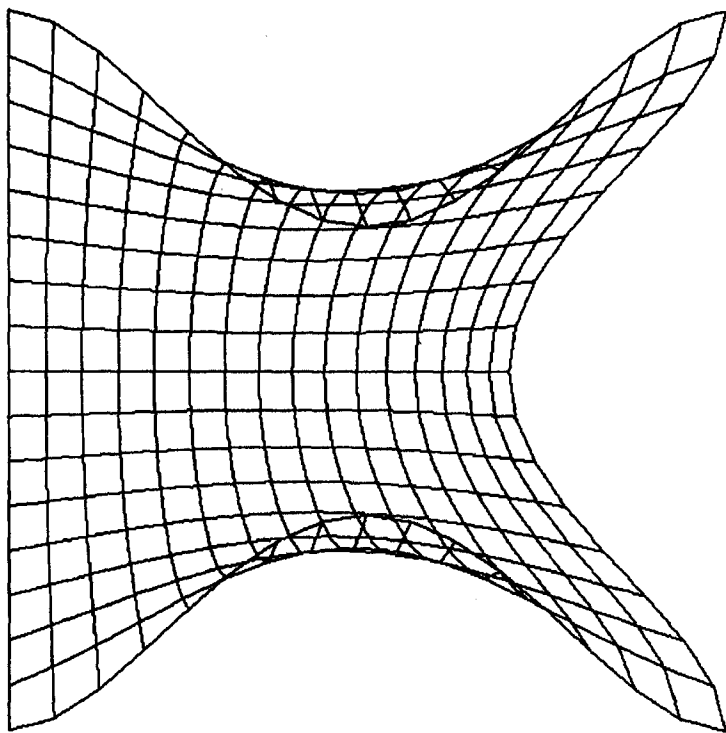


Figure A10. Rectangular reference grid ($A = 1$, $B = 2$) for the fishtail.

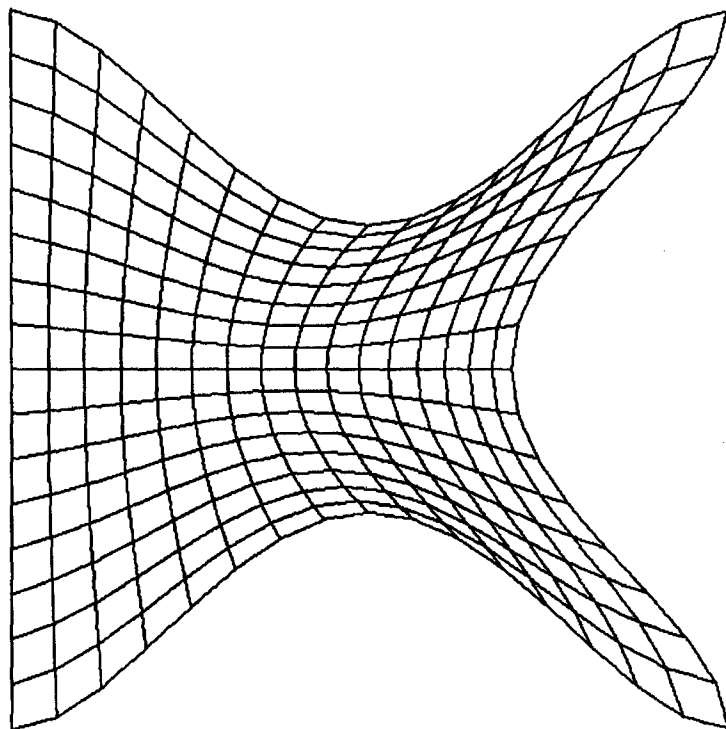


Figure A11. Distinct reference grids for the fishtail.

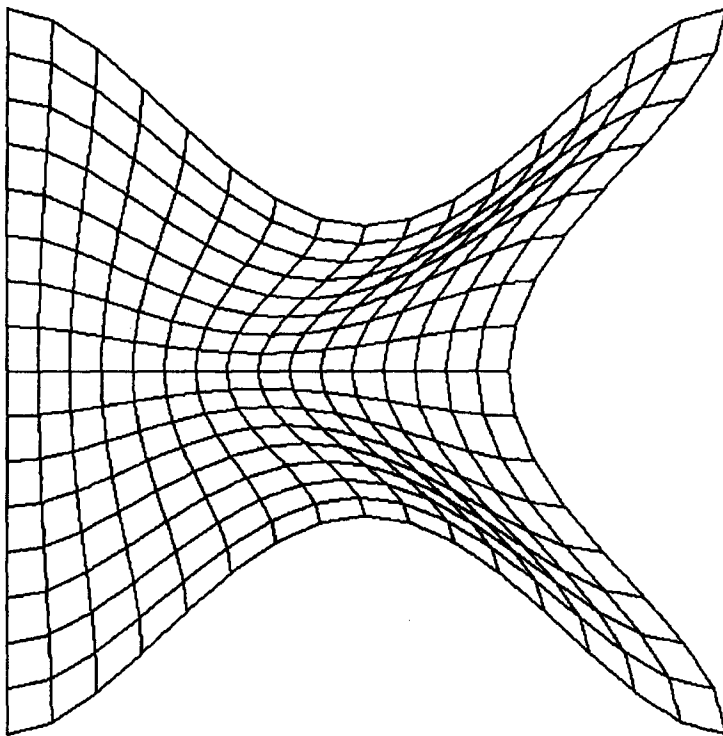


Figure A12. Transfinite interpolation for the **fishtail**.

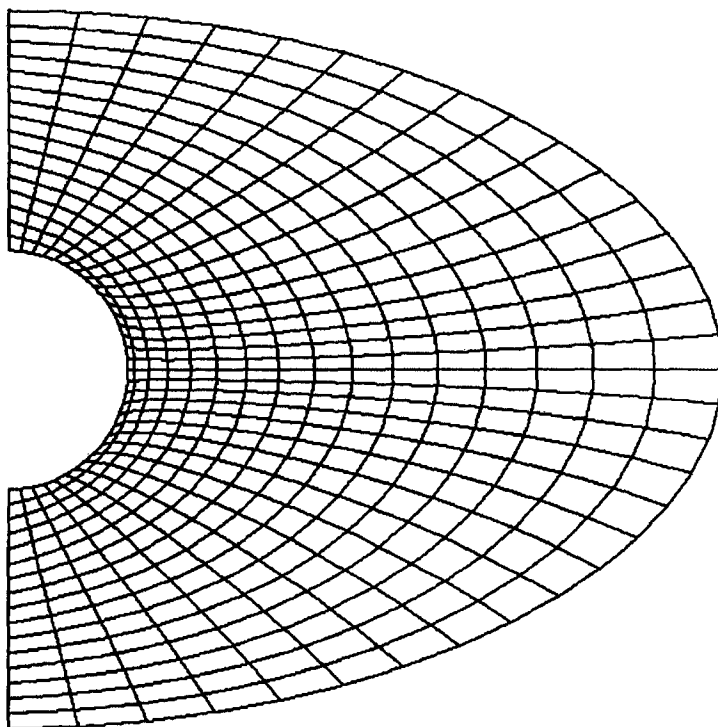


Figure A13. Length weighted by interpolation reference grid for the **ear**.

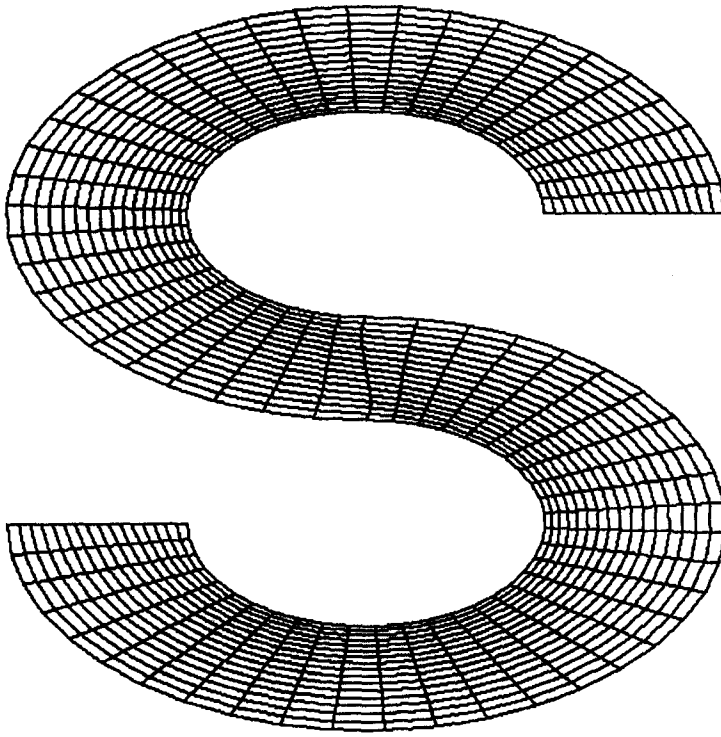


Figure A14. Length weighted by interpolation reference grid for the S.

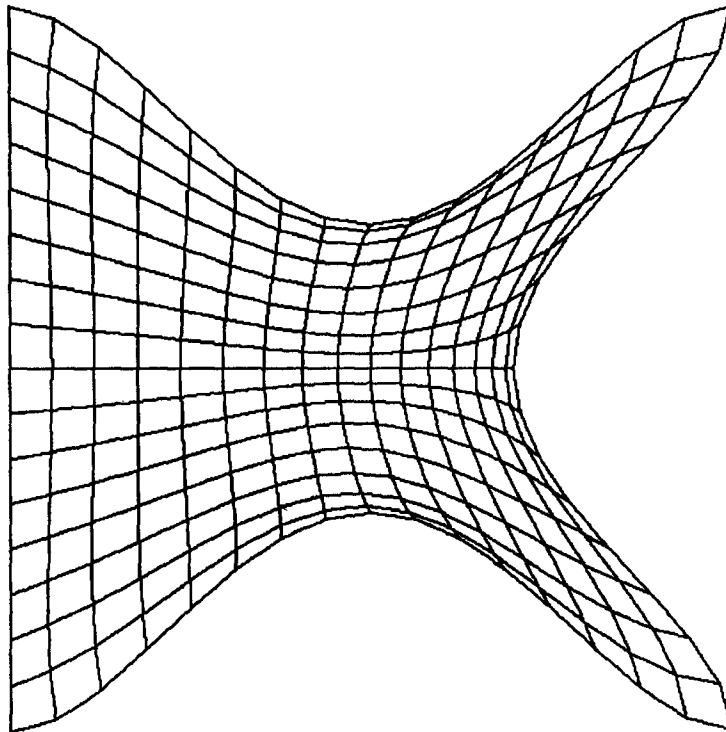
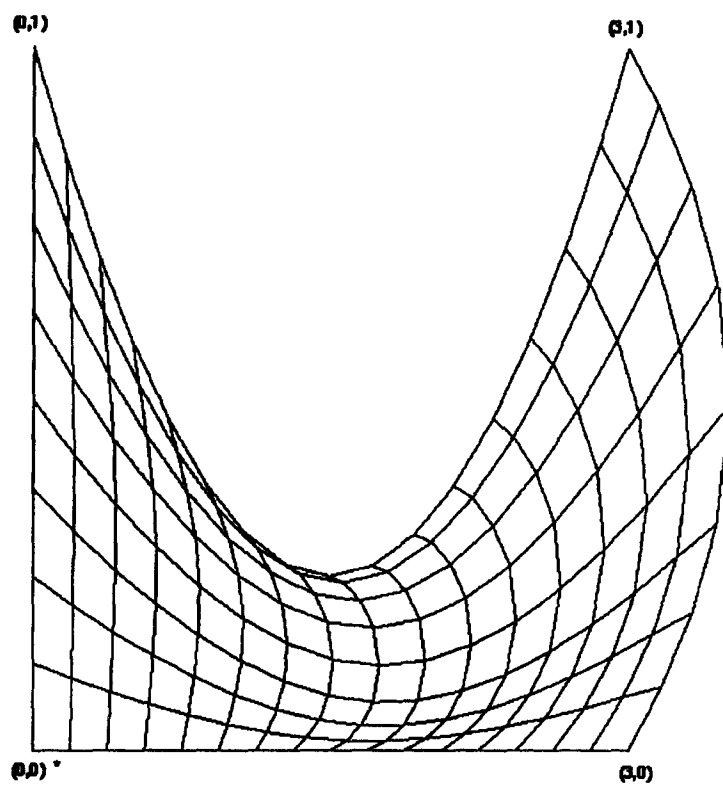
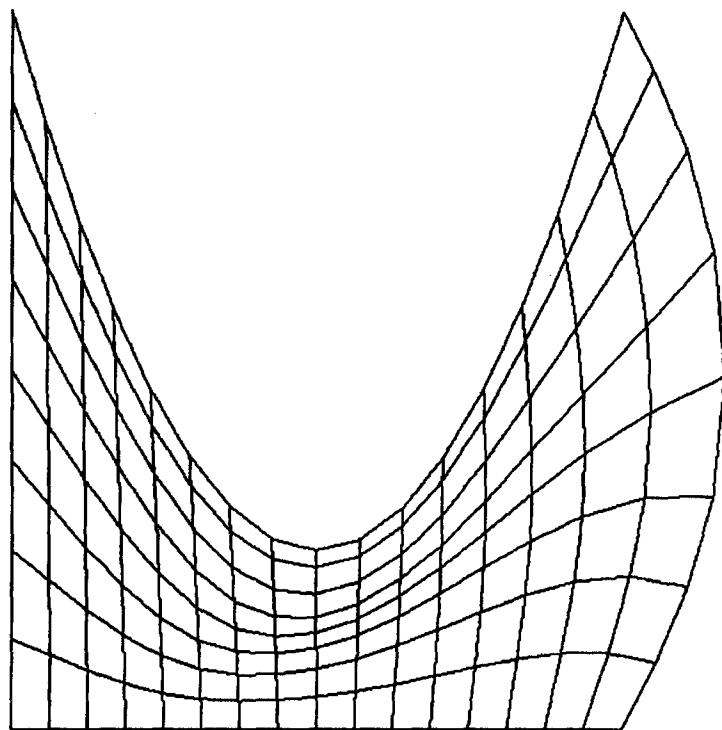


Figure A15. Length weighted by interpolation reference grid for the **fishtail**.

Figure A16. Interpolation grid for the *swan*.Figure A17. Length weighted by interpolation reference grid for the *swan*.

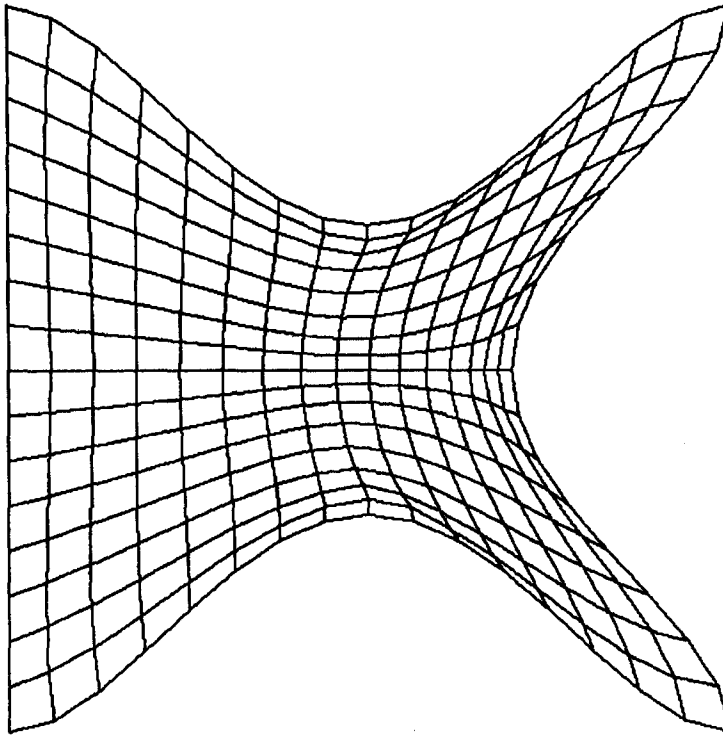


Figure A18. Length with modified right-hand side for the **fishtail**.

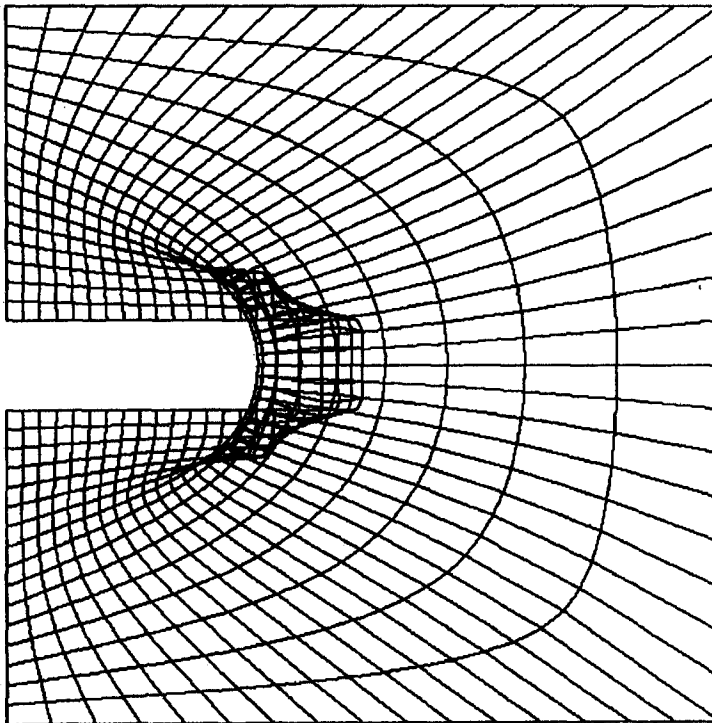


Figure A19. Length with trivial reference grid and nontrivial target set for the **C**.

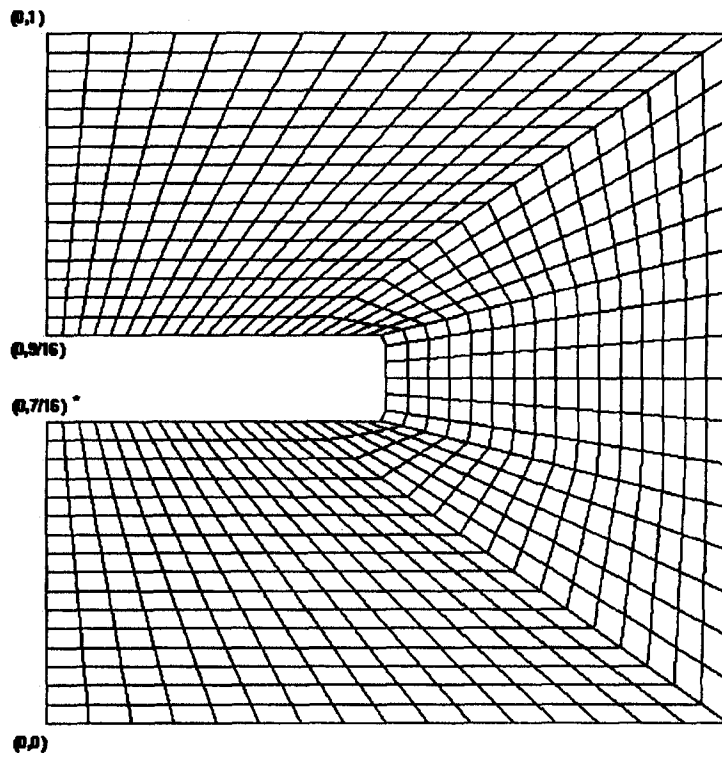


Figure A20. Transfinite interpolation for the C.

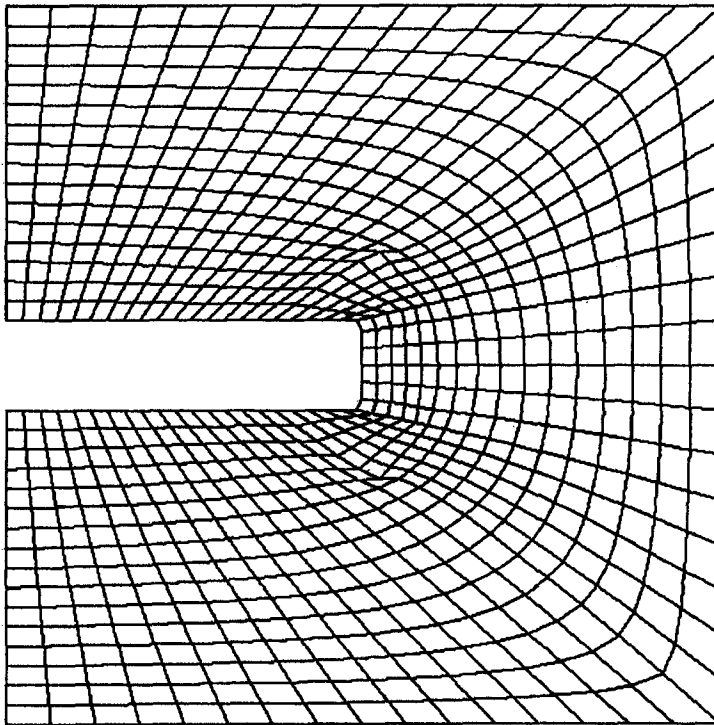


Figure A21. Generalized length functional for the C.

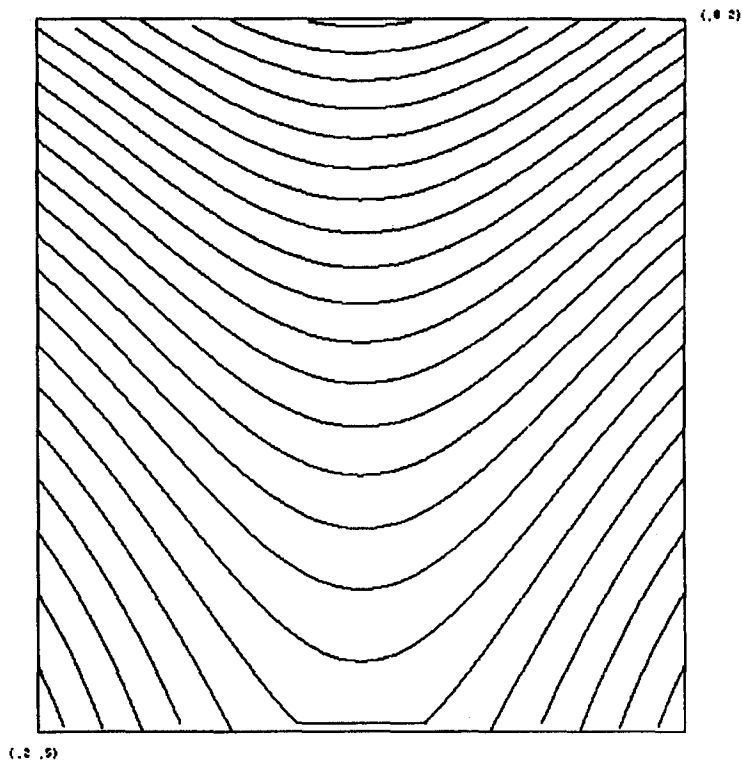


Figure A22. Streamlines for alignment problem.

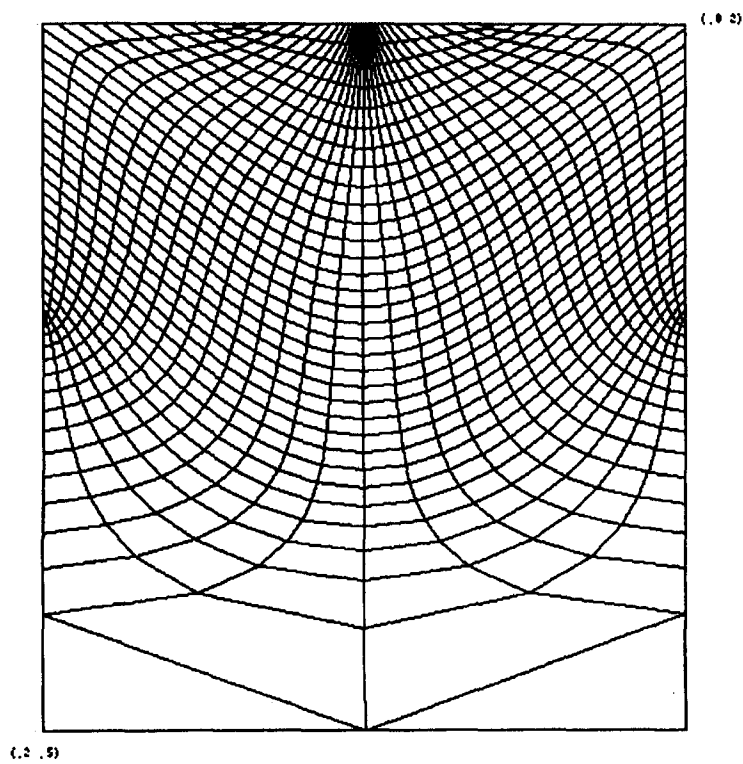


Figure A23. Grid with projection to boundaries solely in terms of vector field.

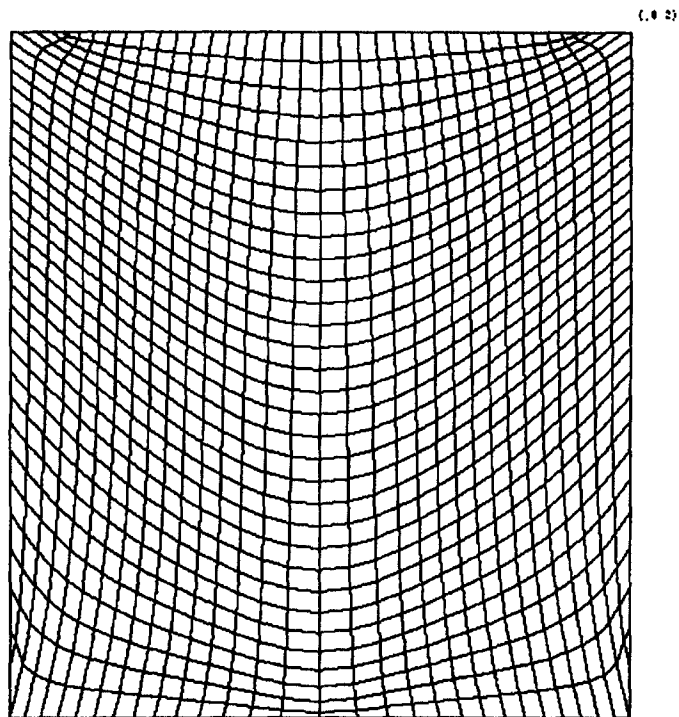


Figure A24. Grid with weighted projection and correction of cell areas near south boundary.

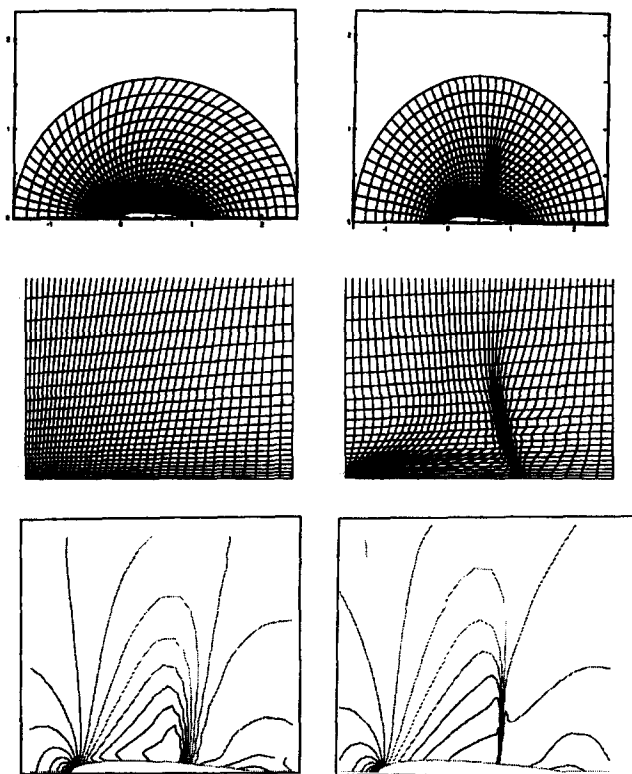


Figure A25. Flowfield feature adaption.